

Machine Learning Models with Hyperparameter Tunning for UCI Adult, Wisconsin Breast Cancer, and Fashion MNIST Datasets.

The team members are:

- Mansi Patel (MS student, MS TEM) - ID 1227152773
- Javier Perez Torres (MS student, MS ECEE) - ID 1217901542 [Team Lead]
- Abhinav Rawat (MS student, MS SCAI) - ID 1231179273

1. Introduction

1.1. Datasets

- 1.1.1. **Wisconsin breast cancer dataset (categorical).** This dataset includes 30 features that describe the characteristics of the cell nucleus and a target column, diagnosis, that shows if the characteristics correspond to a malignant (M) or benign(B) tumor.
- 1.1.2. **The UCI adult dataset (tabular).** This dataset includes 14 features and 48842 instances. This dataset allows training a model to predict whether a person's income exceeds \$50k/yr based on census data.
- 1.1.3. **The Fashion MNIST (numerical).** This dataset contains a training set of 60000 examples and a test set of 10000 examples. Each example is a 28x28 grayscale image associated with a label from 10 classes for a total of 784 pixels in total.

1.2. Model Classes

- 1.2.1. **Logistic Regression:** A linear model used for binary classification. Classify instances into one of two classes based on the probability and the learned decision boundaries. Regularization can be added by applying the L1 or L2 norm to prevent overfitting.
- 1.2.2. **Support Vector Machines:** Seek to find the best hyperplane that separates the data into different classes. Therefore, classifies instances based on their position with respect to the optimal hyperplane. Kernels facilitate transforming the input space to find nonlinear boundaries.
- 1.2.3. **PCA + k-NN:** This approach reduces the dimensionality of the data through PCA, and the instances are classified based on the majority class corresponding to the k-nearest neighbors in the reduced space.
- 1.2.4. **FNN/CNN:** Both are types of neural networks suitable for complex mapping for classification. FNN processes input through layers of neurons with activation functions. CNN uses convolutional layers.

1.3. Team Contribution

- 1.3.1. **Mansi Patel:** Responsible for the SVM model for all datasets. Included assigned model results in the report.
- 1.3.2. **Javier Perez Torres:** Responsible for the Logistic Regression model and PCA + k-NN model for all datasets. Included assigned model results in the report. Created the report's template and introduction. Submission.
- 1.3.3. **Abhinav Rawat:** Responsible for the FNN/CNN model for all datasets. Included assigned model results in the report.

2. UCI Adult

2.1. Logistic Regression with/without Regularization

- Hyperparameters under no regularization: 'alpha': [0.00001, 0.0001, 0.001, 0.01, .1], 'penalty': [None], 'max_iter': [1000], 'loss' ['log_loss'], 'learning_rate' ['constant'], 'eta0' [0.00001, 0.0001, 0.001, 0.01, .1].
- Hyperparameters under regularization: 'alpha': [0.00001, 0.0001, 0.001, 0.01, .1], 'penalty': ['l1', 'l2'], 'max_iter': [1000], 'loss' ['log_loss'], 'learning_rate' : ['constant'], 'eta0' [0.00001, 0.0001, 0.001, 0.01, .1]
- The performance for the 25 and 50 models can be shown in the box plots in Figure 1 in the Appendix under the Logistic Regression Section for the UCI Dataset.
- The best model was found based on accuracy under the no regularization model with the following parameters: Best model: {'alpha': 1e-05, 'eta0': 0.01, 'learning_rate': 'constant', 'loss': 'log_loss', 'max_iter': 1000, 'penalty': None}
- The training and validation log loss and validation accuracy over 20 epochs for this model using those parameters under the LR with no regularization are shown in the Appendix under the Logistic Regression Section for the UCI Dataset in Figure 2.

Evaluating the best model in the test set:

- Testing Accuracy:

Epochs	20	80
Regularization	0.8501574672633847	0.8499917122492956
No Regularization	0.8503232222774739	0.8499917122492956

- Classification Report(best model):

	precision	recall	f1-score	support
0	0.88	0.92	0.90	4503
1	0.74	0.64	0.68	1530
accuracy			0.85	6033
Macro avg.	0.81	0.78	0.79	6033
Weighted avg.	0.85	0.85	0.85	6033

Confusion Matrix(best model):

4156	347
556	974

The ROC curve of the best model is shown in the Appendix under the Logistic Regression Section for the UCI Dataset in Figure 3.

2.2. SVM with/without Kernels

- Hyperparameters using kernel: 'alpha': [0.0001, 0.001], 'penalty': ['l1', 'l2'], 'max_iter': [1000], 'loss' : ['hinge'], 'learning_rate' : ['constant'], 'eta0' : [0.0001, 0.001, 0.01]
- Hyperparameters using no kernel: 'alpha': [0.0001, 0.001], 'penalty': ['l1', 'l2'], 'max_iter': [1000], 'loss' : ['hinge'], 'learning_rate' : ['constant'], 'eta0' : [0.0001, 0.001, 0.01]
- Validation Approach for Hyperparameter Tuning: Grid search with 5-fold cross-validation (cv=5) is used to find the best hyperparameters.

- Plots of Results: Boxplot showing cross-validation scores for each model with different hyperparameters. Two subplots for training and validation:
 - Left: Hinge loss over epochs
 - Right: Accuracy over epochs

Evaluating the best model in the test set:

- Testing Accuracy:

Epochs	20	80
Using Kernel	0.8450190618266202	0.8499917122492956
No Kernel	0.8415382065307475	0.8499917122492956

- Classification Report(best model):

	precision	recall	f1-score	support
0	0.87	0.94	0.90	4503
1	0.76	0.57	0.65	1530
accuracy			0.85	6033
Macro avg.	0.81	0.75	0.78	6033
Weighted avg.	0.84	0.85	0.84	6033

2.3. PCA+k-NN

Hyperparameters: PCA Components [5,50,75], n_neighbors: [3,5,7,9], 'weights': ['uniform','distance'], 'p': [1,2].

The best performance for the 16 models can be shown in the box plots in Figure # in the Appendix under the PCA+k-NN section for the Breast Dataset.

The best model was found based on accuracy under PCA = 50 with the following parameters: Best model: {'n_neighbors': 9, 'p': 1, 'weights': 'uniform'}

Evaluating the best model in the test set:

- Testing Accuracy:

PCA	5	50	75
Best Model	0.8279462953754351	0.836399801093983	0.836399801093983

- Classification Report (best model):

	precision	recall	f1-score	support
0	0.88	0.91	0.89	4503
1	0.70	0.62	0.66	1530
accuracy			0.84	6033
Macro avg.	0.79	0.77	0.78	6033

Weighted avg.	0.83	0.84	0.83	6033
---------------	------	------	------	------

Confusion Matrix (best model):

4094	409
578	952

- A plot of the Explained Variance Ratio for each PCA is shown in the Appendix under the Logistic Regression Section for Breast Dataset

2.4. FNN

- Hyperparameters used - 'lr': [0.001, 0.01, 0.1], 'hidden_sizes': [[128, 64, 16], [32, 16], [64, 16], [64, 8]], 'input_size': [X_train.shape[1]], 'output_size': [1], 'criterion': [nn.BCELoss()], 'optimizer_class': [optim.Adam], 'wd': [0.001, 0.01], 'num_epochs': [50], 'batch_size': [X_train.shape[0]], 'model_class': [FNN]
- The best model worked for the following parameters - {'batch_size': 24129, 'criterion': BCELoss(), 'hidden_sizes': [64, 16], 'input_size': 104, 'lr': 0.01, 'model_class': <class '__main__.FNN'>, 'num_epochs': 50, 'optimizer_class': <class 'torch.optim.adam.Adam'>, 'output_size': 1, 'wd': 0.001}
- The model ran best for BGD as m-BGD and SGD overtrained the model and gave worse results
- The performance for the 24 models can be shown in the box plots in Figure # in the Appendix under the FNN Section for the UCI Dataset.
- The training and validation log loss and validation accuracy over 50 epochs for this model using those parameters is shown in the Appendix under the FNN Section for the UCI Dataset in Figure #.
- Classification Report(best model):
As we can see in Figure # in the Appendix section under UCI adult, we get an F1 score of 0.85 on the best model

Breast Cancer Dataset

2.5. Logistic Regression with/without Regularization

- Hyperparameters under no regularization: 'alpha': [0.0001, 0.001, 0.01, 0.1, .1], 'penalty': [None], 'max_iter': [1000], 'loss' ['log_loss'], 'learning_rate' ['constant'], 'eta0' [0.0001, 0.001, 0.01, 0.1].
 - Hyperparameters under regularization: 'alpha': [0.0001, 0.001, 0.01, 0.1, 1], 'penalty': ['l1', 'l2'], 'max_iter': [1000], 'loss' ['log_loss'], 'learning_rate' : ['constant'], 'eta0' [0.0001, 0.001, 0.01, 0.1]
 - The performance for the 20 and 40 models can be shown in the box plots in Figure in the Appendix under the Logistic Regression section for the Breast Dataset.
 - The best model was found based on accuracy under the regularization model with the following parameters: Best model: {'alpha': 0.001, 'eta0': 0.01, 'learning_rate': 'constant', 'loss': 'log_loss', 'max_iter': 1000, 'penalty': 'l1'}
 - The training and validation log loss and validation accuracy over 5 epochs for this model using those parameters under the LR with regularization are shown in the Appendix under the Logistic Regression Section for the Breast Dataset in Figure #.
- Evaluating the best model in the test set:
- Testing Accuracy:

Epochs	5	8	10
Regularization	0.9912280701754386	0.9824561403508771	0.9824561403508771
No Regularization	0.9824561403508771	0.9824561403508771	0.9824561403508771

- Classification Report (best model):

	precision	recall	f1-score	support
0	0.99	1.00	0.99	71
1	1.00	0.98	0.99	43
accuracy			0.99	114
Macro avg.	0.99	0.99	0.99	114
Weighted avg.	0.99	0.99	0.99	114

Confusion Matrix (best model):

71	0
1	42

The ROC curve of the best model is shown in the Appendix under the Logistic Regression Section for the UCI Dataset in Figure #.

2.6. SVM with/without Kernels

- Hyperparameters using Kernel: 'alpha': [0.0001, 0.01], 'penalty': ['l1', 'l2'], 'max_iter': [1000], 'loss': ['hinge'], 'learning_rate': ['constant'], 'eta0': [0.0001, 0.001, 0.01]
- Hyperparameters without Kernel: 'alpha': [0.0001, 0.001, 0.01, 0.1, 1], 'penalty': ['l1', 'l2'], 'max_iter': [1000], 'loss': ['log_loss'], 'learning_rate': ['constant'], 'eta0' [0.0001, 0.001, 0.01, 0.1]
- The performance for the 20 and 40 models can be shown in the box plots in Figure # in the Appendix under the SVM section for the Breast Dataset.
- The best model was found based on accuracy under the SVM model with the following parameters: Best model: {'alpha': 0.001, 'eta0': 0.01, 'learning_rate': 'constant', 'loss': 'hinge_loss', 'max_iter': 1000, 'penalty': 'l1'}
- The training and validation log loss and validation accuracy over 5 epochs for this model using those parameters under the SVM are shown in the Appendix under theSVM Section for the Breast Dataset in Figure #.

Evaluating the best model in the test set:

- Testing Accuracy:

Epochs	5	8	10
Using Kernel	0.9649122807017544	0.9824561403508771	0.9824561403508771
Without Kernel	0.9824561403508771	0.9824561403508771	0.9824561403508771

- Classification Report (best model):

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.96	0.99	0.97	71
1	0.98	0.93	0.95	43
accuracy			0.96	114
Macro avg.	0.97	0.96	0.96	114
Weighted avg.	0.97	0.96	0.96	114

- Validation Approach for Hyperparameter Tuning: Grid search with 5-fold cross-validation (`cv=5`) using accuracy as the scoring metric.
- Plots of Results: Boxplot for Cross-Validation Scores: X-axis: Model parameters Y-axis: Accuracy, Visualization of cross-validation scores for each combination of hyperparameters. Training and Validation Loss Over Epochs: Two subplots: X-axis: Epoch, Y-axis: Hinge Loss Training and Validation Accuracy Over Epochs: Two subplots: X-axis: Epoch Y-axis: Accuracy

2.7. PCA+k-NN

- Hyperparameters: PCA Components [10,15,20], n_neighbors': [2,3,4,5,6,7,8], 'weights': ['uniform','distance'], 'p': [1,2]
- The best performance for the 28 models can be shown in the box plots in Figure # in the Appendix under the PCA+k-NN section for the Breast Dataset.
- The best model was found based on accuracy under PCA = 10 with the following parameters: Best model: {'n_neighbors': 4, 'p': 1, 'weights': 'distance'}

Evaluating the best model in the test set:

- Testing Accuracy:

PCA	10	15	20
Best Model	0.956140350877193	0.956140350877193	0.956140350877193

- Classification Report (best model):

	precision	recall	f1-score	support
0	0.96	0.97	0.97	71
1	0.95	0.93	0.94	43
accuracy			0.96	114
Macro avg.	0.96	0.95	0.95	114
Weighted avg.	0.96	0.96	0.96	114

Confusion Matrix (best model):

69	2
3	40

- A plot of the Explained Variance Ratio for each PCA is shown in the Appendix under the Logistic Regression Section for Breast Dataset in Figure #.

2.8. FNN

- Hyperparameters used - 'lr': [0.01, 0.001], 'hidden_sizes': [[32, 16], [64, 16], [64, 8]], 'input_size': [X_train.shape[1]], 'output_size': [1], 'criterion': [nn.BCELoss()], 'optimizer_class': [optim.Adam], 'wd': [0.001, 0.01], 'num_epochs': [50], 'batch_size': [32, X_train.shape[0]], 'model_class': [FNN]
The best model worked for the following parameters - {'batch_size': 455, 'criterion': BCELoss(), 'hidden_sizes': [64, 8], 'input_size': 30, 'lr': 0.01, 'model_class': <class '__main__.FNN'>, 'num_epochs': 50, 'optimizer_class': <class 'torch.optim.adam.Adam'>, 'output_size': 1, 'wd': 0.001}
- The performance for the 24 models can be shown in the box plots in Figure # in the Appendix under the FNN Section for the UCI Dataset.
- The training and validation log loss and validation accuracy over 50 epochs for this model using those parameters is shown in the Appendix under the FNN Section for the UCI Dataset in Figure #.
- Classification Report(best model):
As we can see in Figure # in the Appendix section under Wisconsin Breast Dataset, we get an F1 score of 0.85 on the best model

3. Fashion MNIST Dataset

3.1. Logistic Regression with/without Regularization

- For this model, PCA has been added to select the features of the 50 components with more information. A plot of the Explained Variance Ratio for each PCA is shown in the Appendix under the Logistic Regression Section for Fashion MNIST in Figure #.
- Hyperparameters under no regularization: 'alpha': [0.00001, 0.0001, 0.001, 0.01], 'penalty': [None], 'max_iter': [1000], 'loss' ['log_loss'], 'learning_rate' ['constant'], 'eta0' [0.0001, 0.001, 0.01, 0.1].
- Hyperparameters under regularization: 'alpha': [0.00001, 0.0001, 0.001, 0.01], 'penalty': ['l1', 'l2'], 'max_iter': [1000], 'loss' ['log_loss'], 'learning_rate': ['constant'], 'eta0' [0.0001, 0.001, 0.01, 0.1]
- The performance for the 16 and 32 models can be shown in the box plots in Figure in the Appendix under the Logistic Regression section for the Fashion MNIST.
- The best model was found based on accuracy under the regularization model with the following parameters: Best model: {'alpha': 1e-05, 'eta0': 0.001, 'learning_rate': 'constant', 'loss': 'log_loss', 'max_iter': 1000, 'penalty': 'l1'}
- The training and validation log loss and validation accuracy over 5 epochs for this model using those parameters under the LR with regularization are shown in the Appendix under the Logistic Regression Section for the Fashion MNIST dataset in Figure #.

Evaluating the best model in the test set:

- Testing Accuracy:

Epochs	60	80
Regularization	0.8073	0.8079
No Regularization	0.8067	0.8076

- Classification Report (best model) can be shown in the Appendix under the Logistic Regression Section for the Fashion MNIST dataset in Table #

3.2. SVM with/without Kernels

For this model, PCA has been added to select the features of the 50 components with more information. A plot of the Explained Variance Ratio for each PCA is shown in the Appendix under the Logistic Regression Section for Fashion MNIST in Figure #.

Hyperparameters using kernel: 'alpha': [0.00001, 0.0001, 0.001, 0.01], 'penalty': [None], 'max_iter': [1000], 'loss' ['log_loss'], 'learning_rate' ['constant'], 'eta0' [0.0001, 0.001, 0.01, 0.1].

Hyperparameters without using kernel: 'alpha': [0.001, 0.0001, 0.001, 0.01], 'penalty': ['l1', 'l2'], 'max_iter': [1000], 'loss' ['log_loss'], 'learning_rate' : ['constant'], 'eta0' [0.0001, 0.001, 0.01, 0.1]

- The performance for the 16 and 32 models can be shown in the box plots in Figure in the Appendix under the SVM section for the Fashion MNIST.
- The best model was found based on accuracy under the SVM model with the following parameters: Best model: {'alpha': 1e-05, 'eta0': 0.001, 'learning_rate': 'constant', 'loss': 'log_loss', 'max_iter': 1000, 'penalty': 'l1'}
- The training and validation log loss and validation accuracy over 5 epochs for this model using those parameters under the SVM are shown in the Appendix under the SVM Section for the Fashion MNIST dataset in Figure #.

Evaluating the best model in the test set:

- Testing Accuracy:

Epochs	60	80
With kernel	0.8173	0.8178
Without kernel	0.8167	0.8174

3.3. PCA+k-NN

Hyperparameters: PCA Components [5,15,20], n_neighbors': [2,3,4,5,6,7,8], 'weights': ['uniform','distance'], 'p': [1,2]

The best performance for the 28 models can be shown in the box plots in Figure # in the Appendix under the PCA+k-NN section for the Fashion MNIST.

The best model was found based on accuracy under PCA = 10 with the following parameters: Best model: {'n_neighbors': 8, 'p': 1, 'weights': 'distance'}

Evaluating the best model in the test set:

- Testing Accuracy:

PCA	5	15	20
Best Model	0.7451	0.8362	0.8501

- Classification Report (best model) can be shown in the Appendix under the Logistic Regression Section for the Fashion MNIST dataset in Table #
- A plot of the Explained Variance Ratio for each PCA is shown in the Appendix under the Logistic Regression Section for Breast Dataset in Figure #.

3.4. FNN

- Hyperparameters used - 'lr': [0.001,0.01, 0.1], 'hidden_sizes': [[128, 64, 8],[256, 16, 8], [128, 16], [256, 32]], 'input_size': [X_train.shape[1]], 'output_size': [10], 'criterion': [nn.CrossEntropyLoss()],

'optimizer_class': [optim.Adam], 'wd': [0.001, 0.01], 'num_epochs': [50, 80], 'batch_size': [X_train.shape[0]], 'model_class': [FNN]

- The best model worked for the following parameters - {'batch_size': 60000, 'criterion': CrossEntropyLoss(), 'hidden_sizes': [256, 32], 'input_size': 784, 'lr': 0.01, 'model_class': __main__.FNN, 'num_epochs': 50, 'optimizer_class': torch.optim.adam.Adam, 'output_size': 10, 'wd': 0.001}
- The performance for the 48 models can be shown in the box plots in Figure # in the Appendix under the FNN Section for the UCI Dataset.
- The training and validation log loss and validation accuracy over 50 epochs for this model using those parameters is shown in the Appendix under the FNN Section for the UCI Dataset in Figure #.
- Classification Report(best model):
As we can see in Figure # in the Appendix section under Wisconsin Breast Dataset, we get an F1 score of 0.85 on the best model
- Confusion matrix:

We can see in Figure in Appendix section under Wisconsin Breast Dataset the resultant confusion matrix

3.5. CNN

- Hyperparameters used - 'lr': [0.001], 'hidden_sizes': [[32, 48],[32, 64]], 'input_size': [28], 'output_size': [10], 'criterion': [nn.CrossEntropyLoss()], 'optimizer_class': [optim.Adam], 'wd': [0.001, 0.01], 'num_epochs': [5], 'batch_size': [x_train.shape[0]//2], 'model_class': [CNN], 'kernel_size_conv': [3], 'kernel_size_pool': [2], 'padding': [1], 'stride_conv': [1], 'stride_pool': [2]
- The best model worked for the following parameters - 'batch_size': 5000, 'criterion': CrossEntropyLoss(), 'hidden_sizes': [32, 48], 'input_size': 28, 'kernel_size_conv': 3, 'kernel_size_pool': 2, 'lr': 0.001, 'model_class': __main__.CNN, 'num_epochs': 5, 'optimizer_class': torch.optim.adam.Adam, 'output_size': 10, 'padding': 1, 'stride_conv': 1, 'stride_pool': 2, 'wd': 0.001
- The performance for the 4 models can be shown in the box plots in Figure # in the Appendix under the CNN Section for the UCI Dataset.
- The training and validation log loss and validation accuracy over 50 epochs for this model using those parameters is shown in the Appendix under the CNN Section for the UCI Dataset in Figure #.
- Classification Report(best model):
As we can see in Figure # in the Appendix section under Fashion MNIST Dataset, we get an F1 score of 0.85 on the best model
- Confusion matrix:

We can see in Figure # in Appendix section under Fashion MNIST Dataset the resultant confusion matrix

4. Value of Concepts Learned in the Course

The concepts learned in the course of Statistical Machine Learning (SML) are invaluable in understanding and leveraging advanced techniques for data analysis and decision-making. The concepts learned in the course of Statistical Machine Learning have proven to be invaluable in understanding and navigating the complex landscape of machine learning algorithms. Topics such as regularization, bias-variance trade offs, and cross-validation have provided essential insights into model performance and robustness. Regularization techniques, including L1 and L2 regularization, have equipped us with tools to prevent overfitting and improve the generalization of models. Understanding the delicate balance between bias and variance has been pivotal in model selection and optimization, ensuring that the models neither underfit nor overfit the data. Additionally, cross-validation methodologies have been instrumental in assessing model generalization across different subsets of data, providing a comprehensive evaluation of predictive performance. These concepts collectively form a solid foundation, guiding the selection, tuning, and evaluation of machine learning models, making the acquired knowledge an indispensable asset in the pursuit of effective and reliable data-driven solutions.

5. Conclusions

After completing this project, the team understood the different approaches when attempting to create a machine learning model. After exploring 3 different datasets and applying different models with different parameters, the team understood that there are types of datasets that can perform better based on the type of data and the amount of information provided. As it was expected the Logistic Regression model worked great for small binary classification datasets, but its accuracy decreased with more numerical and complex datasets. In fact, the team opted for applying PCA to reduce the computational complexity when running the model. For the SVM models, some datasets presented challenges for the mixed data types and the need for effective feature representation. However, SVMs with kernel approximations offered the advantage of handling non-linear relationships by mapping input features into higher-dimensional spaces. On the other hand, PCA allowed the team to reduce the complexity of the model by eliminating components that do not handle too much information from the features allowing for a much faster computation. Combining it with k-NN we obtained a simple and efficient model that correctly predicts based on the complexity of the dataset.

6. References

[1]scikit-learn , “sklearn.neighbors.KNeighborsClassifier — scikit-learn 0.22.1 documentation,” Scikit-learn.org, 2019.

<https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>

[2]“sklearn.linear_model.SGDClassifier — scikit-learn 0.22.2 documentation,” *scikit-learn.org*.

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html

[3] Chat GPT. Utilize to aid in the creation of outliners for the code and to optimize for running efficiency. Utilized to correct grammar in text in the Introduction, Value of Concepts, and Conclusion sections.

Appendix

1. UCI Adult Dataset

1.1. Logistic Regression with/without Regularization

The team utilized Stochastic Gradient Descent from the Scikit Learn package. Under this application, the loss gradient is estimated for each sample at a time, and the model is updated along the way with a learning rate. Here are the parameters that were selected for the SGD Classifier: 'alpha', 'penalty', 'max_iter', 'loss', 'learning_rate', and 'eta0'. The 'loss' is set to log_loss, so the model outputs a logistic regression model. The penalty varies depending on regularization; for regularization models, the L1 norm and L2 norm are used for models without regularization, the penalty is set to None. Alpha is a constant that multiplies the regularization term; the higher the value, the stronger the regularization applied. Max_iter was set to default to 1000. The learning rate was set to constant, meaning that the $\eta = \eta_0$. Lastly, η_0 defines the initial learning rate. The flowchart of the code is as follows: Install Libraries -> Load and Preprocess Dataset -> Standardize Features -> Define Model -> Grid Search for Hyperparameter Tuning -> Cross-Validation with GridSearchCV -> Visualize Cross-Validation Scores -> Train Best Model with Partial Fit and Batch-Wise Training -> Evaluate Training and Validation Losses -> Plot Training and Validation Metrics Over Epochs -> Evaluate Best Model on Test Set -> Display Testing Accuracy, Classification Report, and Confusion Matrix. The code uses the following libraries and packages: TensorFlow (tensorflow): Deep learning library. Scikit-learn (sklearn): Machine learning library. NumPy (numpy): Numerical computing library. Matplotlib (matplotlib): Plotting library. Seaborn (seaborn): Statistical data visualization library. Pandas (pandas): Data manipulation and analysis library. The code uses the following metrics: Accuracy Score, Log Loss, Classification Report and the Confusion Matrix.

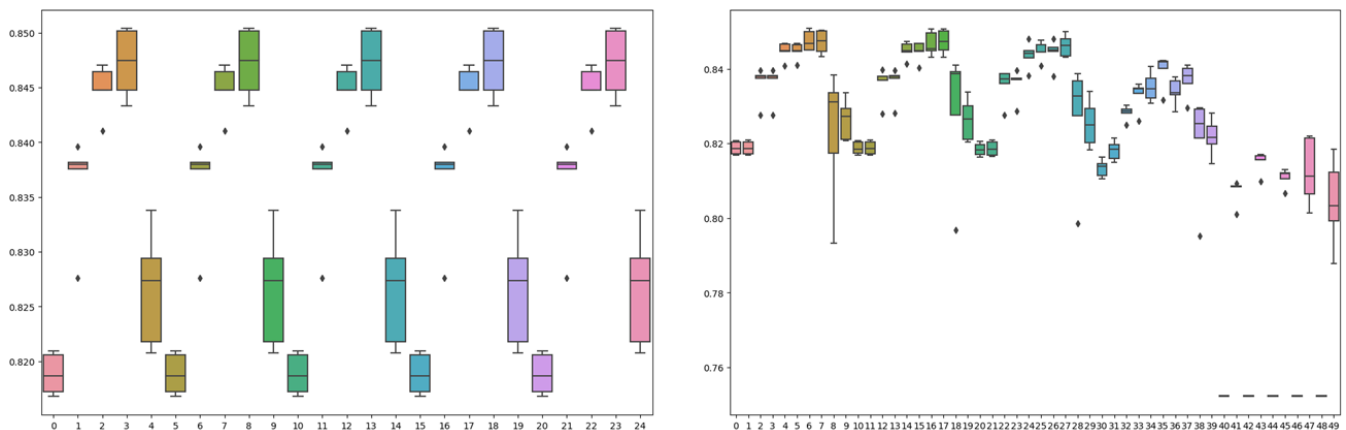


Figure 1. Box plot for the Logistic Regression without Regularization for 25 models under the UCI Adult Training Dataset and box plot for the LR_Regularization Logistic Regression with Regularization 50 models under the UCI Adult Training Dataset

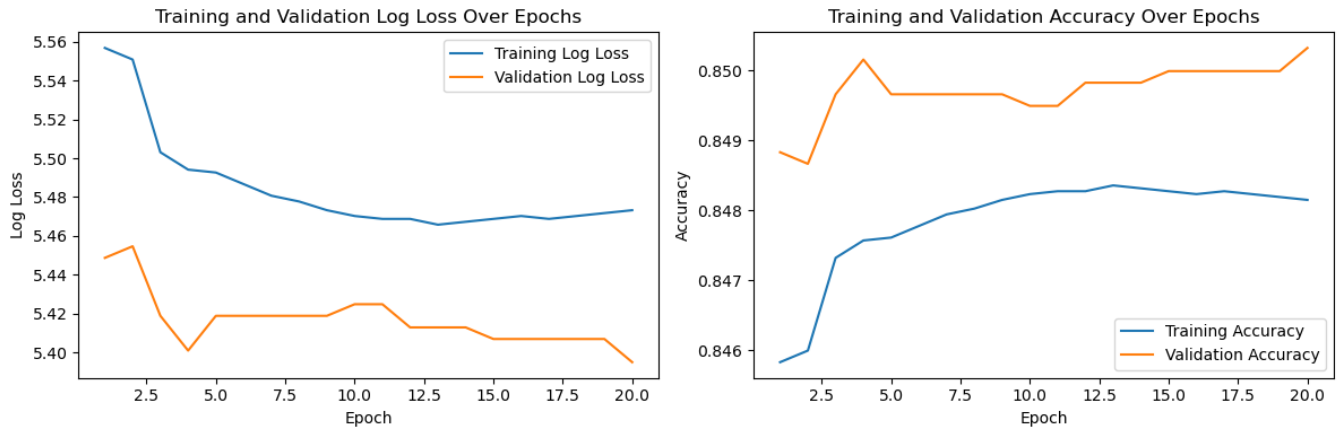


Figure 2. Training and Validation Log Loss Over 20 Epochs and Training and Validation Accuracy Over 20 Epochs for the Logistic Regression without Regularization on the best model under the UCI Adult Dataset.

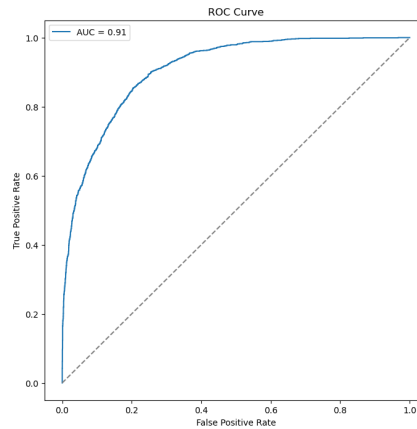


Figure 3. ROC Curve for the Logistic Regression without Regularization best model of the UCI Adult Dataset.

1.2. SVM with/without Kernels

Libraries Used: matplotlib.pyplot, seaborn, numpy, sklearn modules: model_selection: GridSearchCV, cross_val_score, train_test_split, linear_model: SGDClassifier, kernel_approximation: RBFSampler preprocessing: StandardScaler, datasets: load_, metrics: accuracy_score, classification_report, roc_curve, roc_auc_score, hinge_loss, log_loss. Flowchart of the Process: Data Loading and Preprocessing → Kernel Approximation → Model Initialization → Hyperparameter Grid Definition → Batch-wise Training Simulation → Performance Metrics Calculation → Plotting Results → Grid Search for Hyperparameter Tuning → Cross-Validation Scores Visualization → Retrain Best Model with Batch-wise Training → Evaluation on Test Set

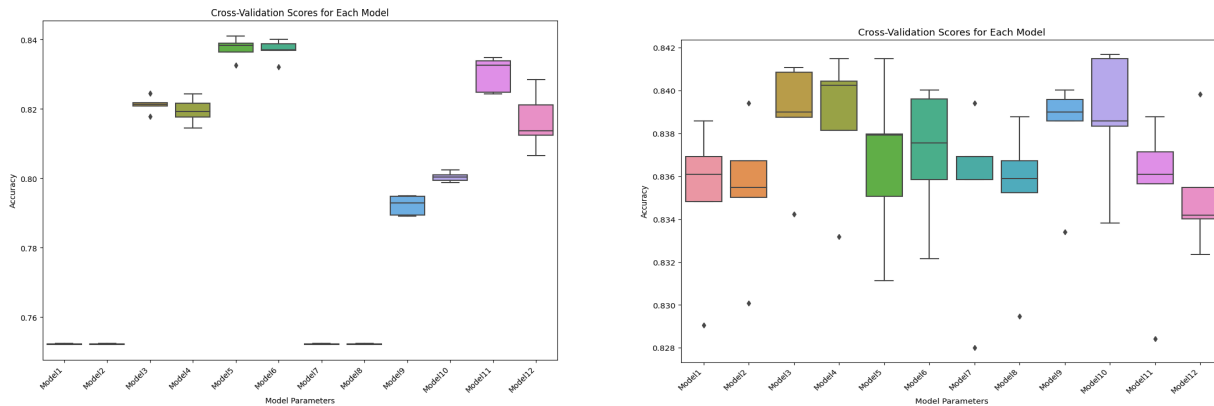


Figure 4. Box plot for the SVM with/without kernel for 25 models and 50 models under the UCI Adult Training Dataset

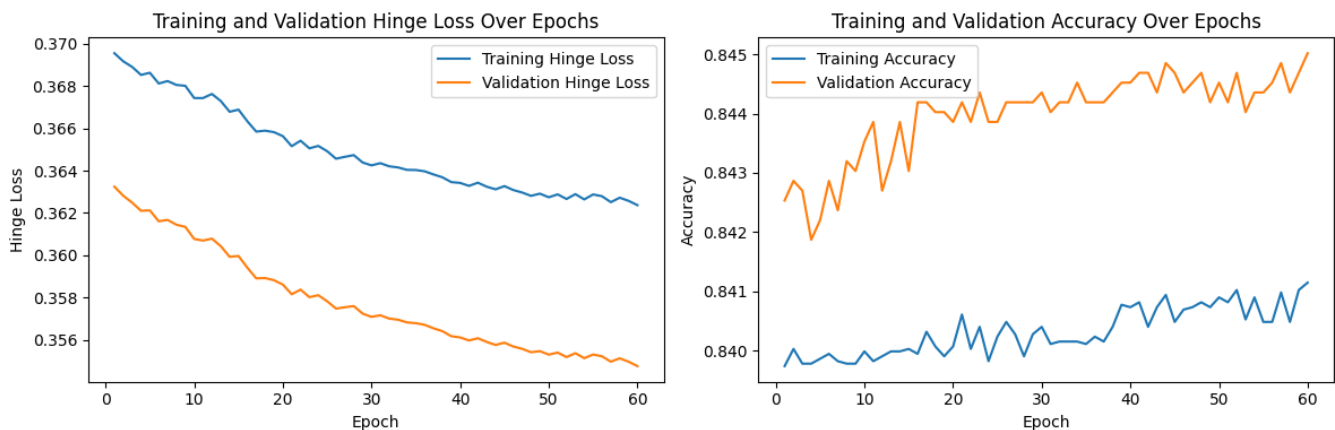


Figure 5. Training and Validation Hinge Loss Over 20 Epochs and Training and Validation Accuracy Over 20 Epochs for the SVM using and not using Kernel on the best model under the UCI Adult Dataset.

1.3. PCA + k-NN

The team utilized k-nearest neighbors and Principal Component Analysis from the Scikit Learn package. Here are the parameters that were selected for the k-NN classifier: `n_neighbors`, `weights`, and `p`. The 'loss' is calculated from `log_loss`. The weights are uniform, which weights all points equally, and distance, which weights points by the inverse of their distance. When `p = 1`, this is equivalent to using

manhattan_distance (l1), and euclidean_distance (l2) for $p = 2$. Install Libraries -> Load and Preprocess Dataset -> Standardize Features -> Determine Maximum Number of Components -> Apply PCA for Dimensionality Reduction (for each specified number of components) -> Visualize Principal Components -> Define Model -> Grid Search for Hyperparameter Tuning -> Cross-Validation with GridSearchCV -> Visualize Cross-Validation Scores -> Train Best Model -> Evaluate Best Model on Test Set -> Display Testing Accuracy, Classification Report, and Confusion Matrix. The code uses the following libraries and packages: TensorFlow (tensorflow): Deep learning library. Scikit-learn (sklearn): Machine learning library. NumPy (numpy): Numerical computing library. Matplotlib (matplotlib): Plotting library. Seaborn (seaborn): Statistical data visualization library. Pandas (pandas): Data manipulation and analysis library. The code uses the following metrics: Accuracy Score, Log Loss, Classification Report and Confusion Matrix.

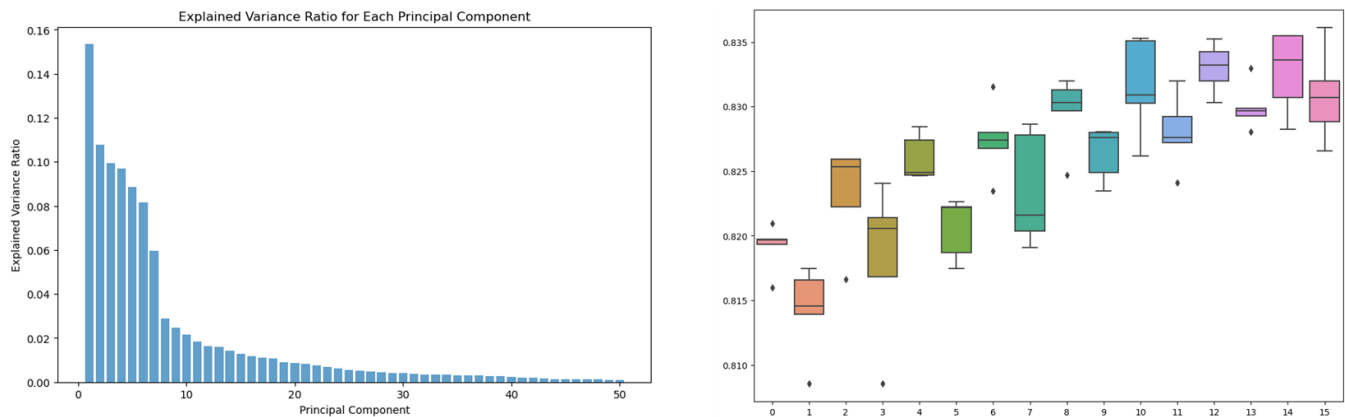


Figure 6. Explained Variance Ratio for Each Principal Component (50) Breast Cancer Dataset for PCA+kNN and Box plot for PCA+kNN for 16 models under the UCI Adult Dataset.

1.4. FNN

The provided Python script showcases a complete pipeline for binary classification on the Breast Cancer Wisconsin Diagnostic dataset using PyTorch. The code begins by loading and preprocessing the dataset, employing standardization and mapping class labels to binary values. Subsequently, a feedforward neural network (FNN) model is defined with customizable hidden layers, activation functions, and dropout. The PyTorch estimator class is then introduced, facilitating training and evaluation of the neural network with specified hyperparameters. A grid search, leveraging GridSearchCV, is performed to discover the optimal hyperparameter configuration through cross-validation. The results are visualized using box plots, offering a comparative analysis of different model configurations. The best model is subsequently trained, and its performance is evaluated on the test set, considering metrics such as accuracy, classification report, and ROC curve with AUC. The code structure is well-organized, employing modular functions, and the use of custom classes enhances code readability and reusability throughout the entire machine learning workflow.

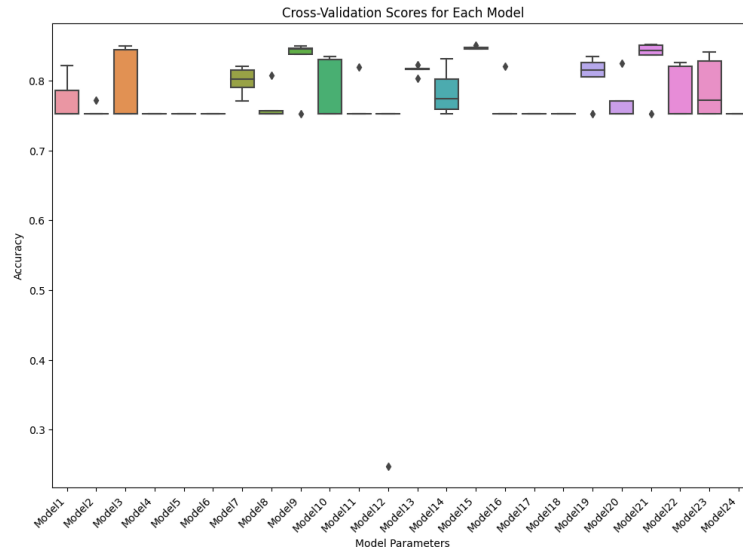


Fig 7 Box Plots for 24 models for FNN

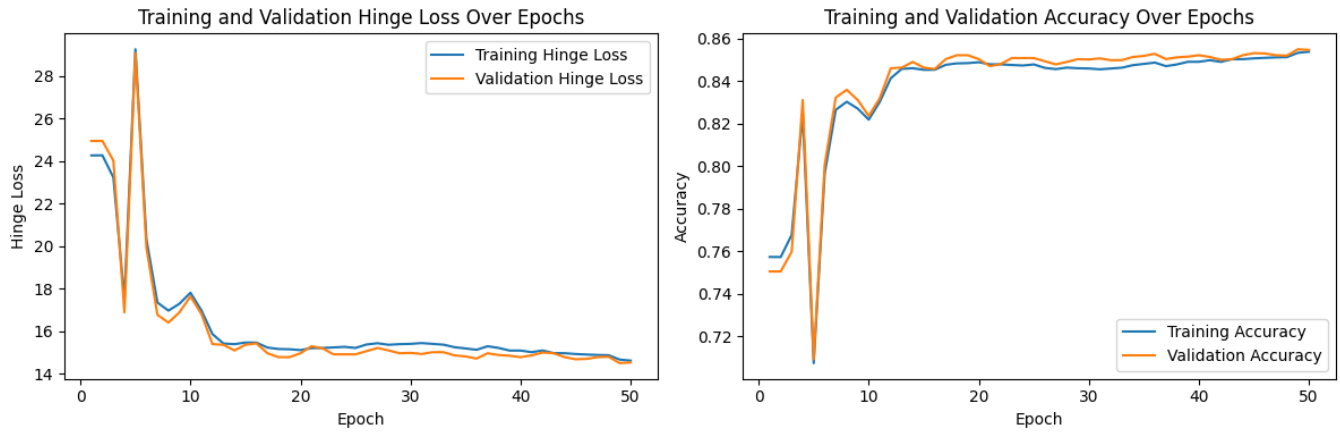


Fig 8 Training and validation losses and Accuracies for best FNN model

```

Testing Accuracy: 0.8546328526437925
Classification Report:

```

	precision	recall	f1-score	support
0.0	0.89	0.92	0.90	4503
1.0	0.74	0.65	0.69	1530
accuracy			0.85	6033
macro avg	0.82	0.79	0.80	6033
weighted avg	0.85	0.85	0.85	6033

Fig 9 Classification report for the best model

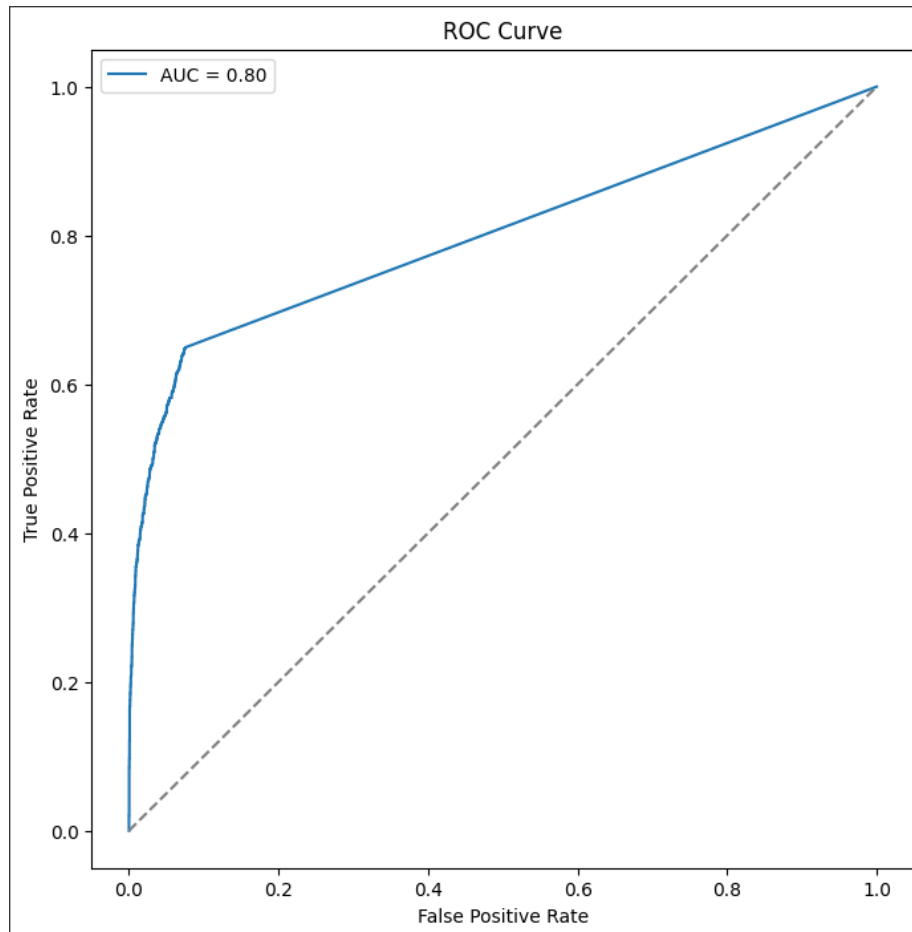


Fig 10 ROC for the best model

2. Wisconsin Breast Cancer Dataset

2.1. Logistic Regression with/without Regularization

The same model application from the UCI Adult Dataset was used for the Wisconsin Breast Cancer dataset for both Logistic Regression with and without regularization.

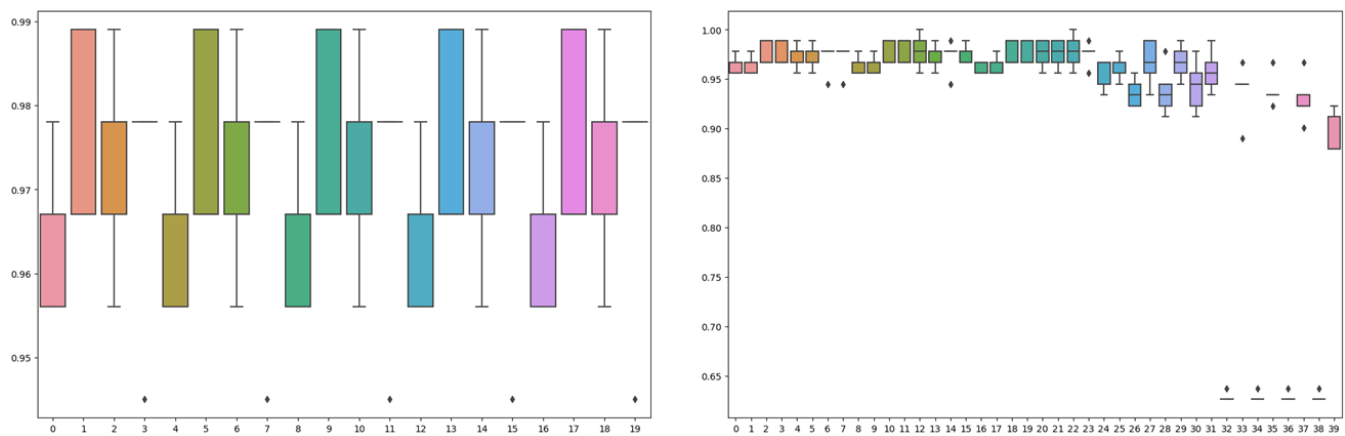


Figure 11 Box plot for the LR_No_Regularization 20 models under the Breast Cancer Training Dataset and Box plot for the LR_Regularization 40 models under the Breast Cancer Training Dataset

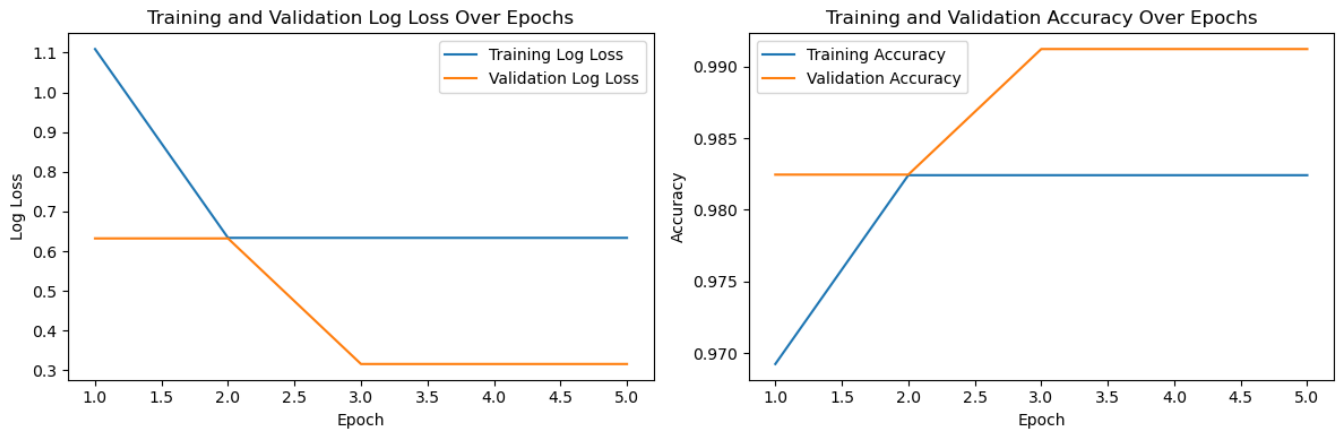


Figure 12 Training and Validation Log Loss Over 5 Epochs and Training and Validation Accuracy Over 5 Epochs for the LR_Regularization best model.

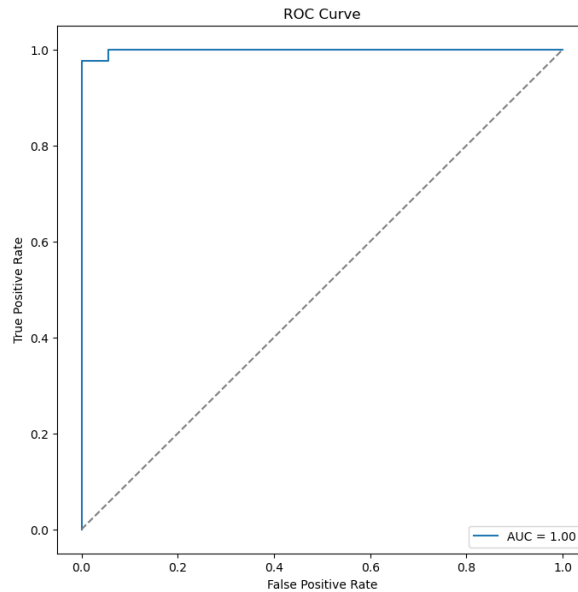


Figure 13 ROC Curve for the LR_Regularization best model of the Breast Cancer Dataset

2.2. SVM with/without Kernels

The same model application from the UCI Adult Dataset was used for the Wisconsin Breast Cancer dataset for both SVM using and not using Kernel.

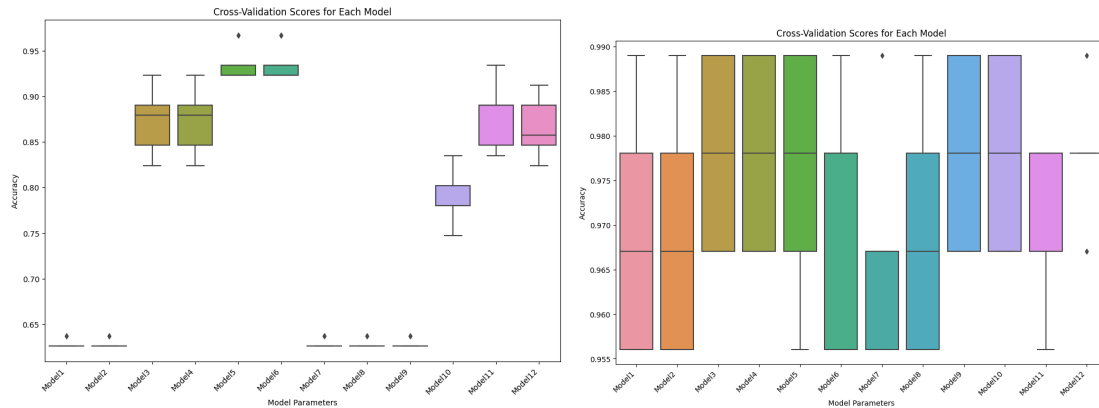


Figure 14 Box plot for the SVM with/without kernel for 20 models and 40 models under the Breast Cancer Training Dataset

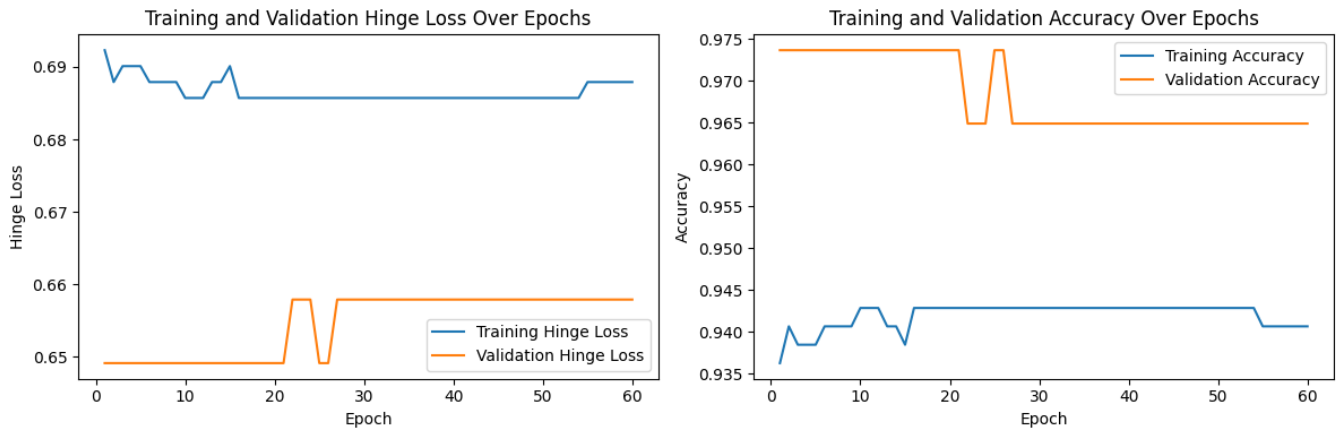


Figure 15 Training and Validation Log Loss Over 5 Epochs and Training and Validation Accuracy Over 5 Epochs for the SVM with and without Kernel best model.

2.3. PCA + k-NN

The same model application from the UCI Adult Dataset was used for the Wisconsin Breast Cancer dataset for PCA+kNN.

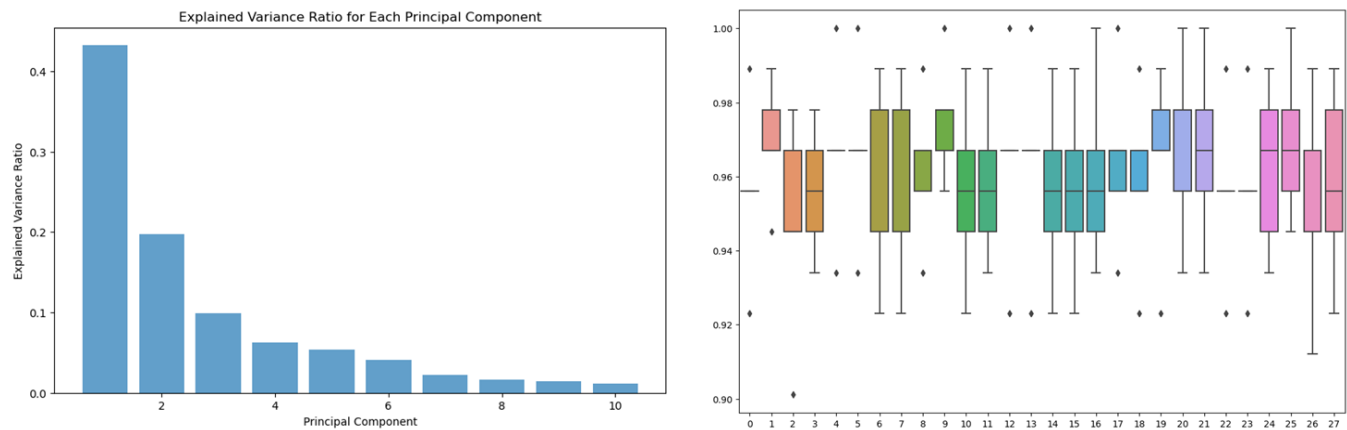


Figure 16 Explained Variance Ratio for Each Principal Component (10) Breast Cancer Dataset for PCA+kNN and Box plot for PCA+kNN for 28 models under the Breast Cancer Dataset.

2.4. FNN

The presented Python script implements a binary classification task on the Breast Cancer Wisconsin Diagnostic dataset using a feedforward neural network (FNN) in PyTorch. The workflow begins with data preprocessing, including standardization and encoding of class labels. The FNN model is defined with customizable architecture, incorporating linear layers, ReLU activations, and dropout for regularization. A PyTorch estimator class is introduced to facilitate model training and evaluation, and a grid search using GridSearchCV is performed to optimize hyperparameters through cross-validation. The script then visualizes the cross-validation results using box plots, allowing a comparative analysis of different model configurations. The best model is selected based on accuracy scores, and its performance is assessed on the test set. Training and validation losses, as well as accuracies, are plotted over epochs, and the final model is evaluated using metrics such as accuracy, a classification report, and an ROC curve with AUC. The code structure is well-organized, leveraging PyTorch and scikit-learn functionalities, providing a comprehensive and modular approach to building and evaluating neural network models for binary classification.

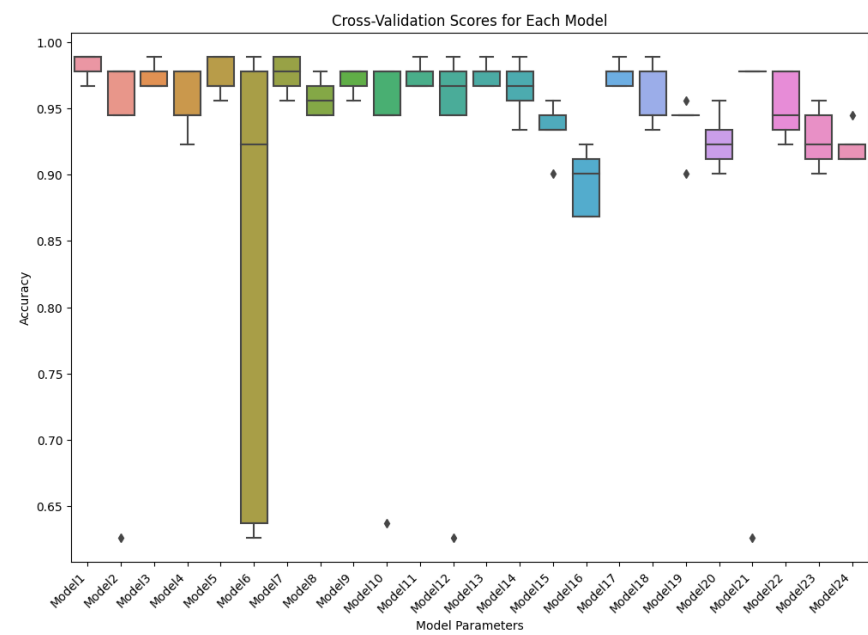


Fig # Box Plots or 24 models of FNN

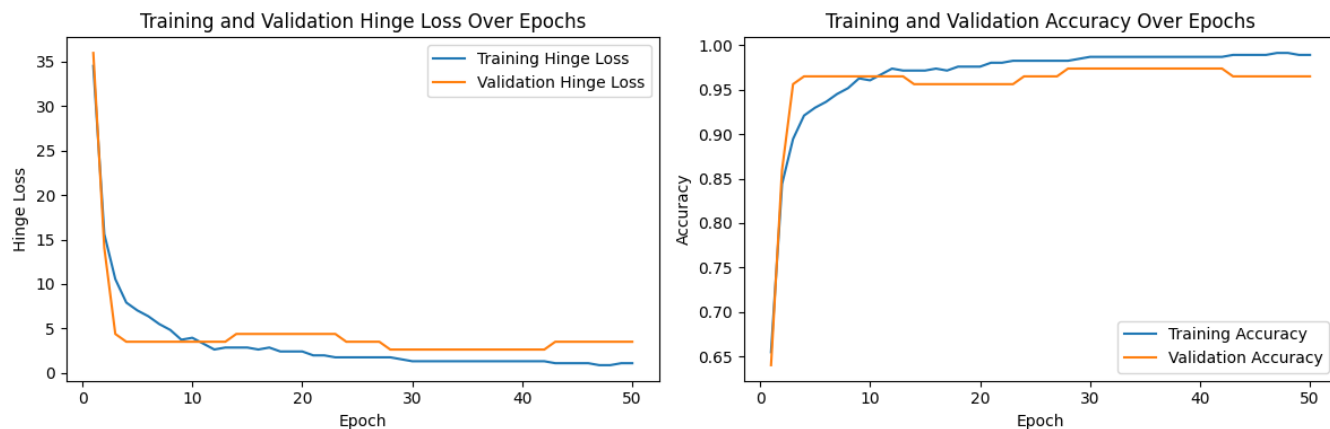


Fig # Training and validation losses and Accuracies for best FNN model

```
Testing Accuracy: 0.9649122807017544
Classification Report:
```

	precision	recall	f1-score	support
0.0	0.99	0.96	0.97	71
1.0	0.93	0.98	0.95	43
accuracy			0.96	114
macro avg	0.96	0.97	0.96	114
weighted avg	0.97	0.96	0.97	114

Fig # Classification report for the best model

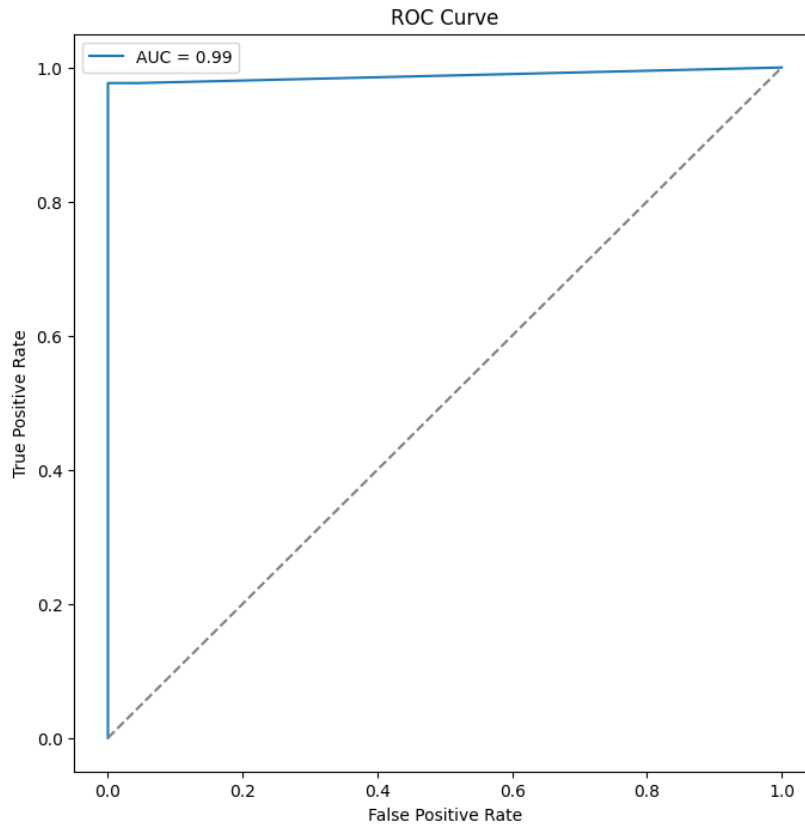


Fig # ROC-AUC curve for the best model

3. Fashion MNIST Dataset

3.1. Logistic Regression with/without Regularization

The same model application from the UCI Adult Dataset was used for the Wisconsin Breast Cancer dataset for both Logistic Regression with and without regularization with the addition of PCA for 50 components.

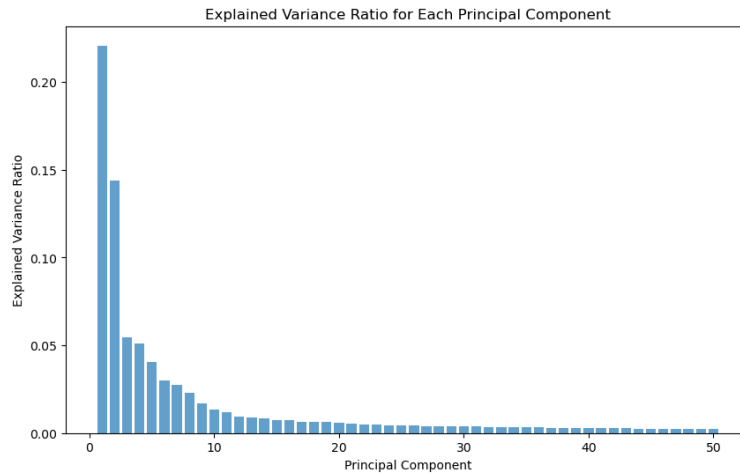


Figure 17 Explained Variance Ratio for Each Principal Component (50) Fashion MNIST for Logistic Regression.

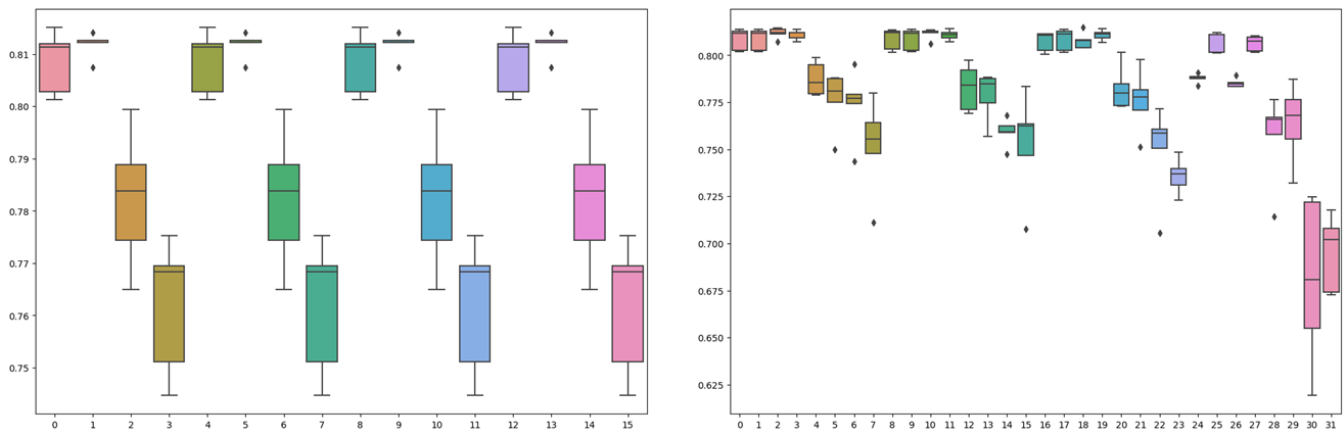


Figure 18 Box plot for the Logistic Regression without regularization for 16 models under the Fashion MNIST Dataset and Box plot for the Logistic Regression with regularization for 32 models under the Fashion MNIST Dataset.

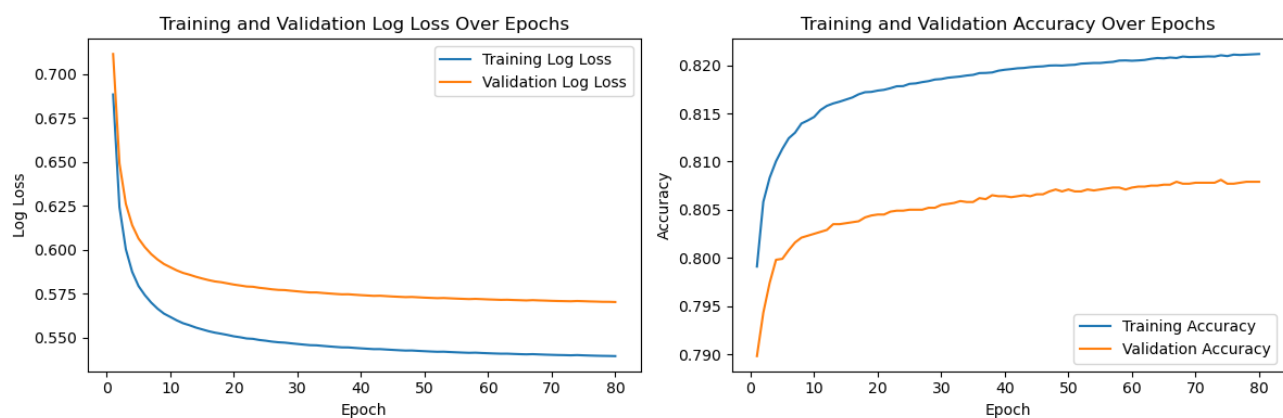


Figure 19 Training and Validation Log Loss Over 80 Epochs and Training and Validation Accuracy Over 80 Epochs for the Logistic Regression with regularization of the best model under the Fashion MNIST Dataset.

	precision	recall	f1-score	support
0	0.76	0.80	0.78	1000
1	0.97	0.95	0.96	1000
2	0.72	0.66	0.69	1000
3	0.80	0.85	0.83	1000
4	0.64	0.79	0.71	1000
5	0.90	0.89	0.89	1000
6	0.53	0.40	0.45	1000
7	0.88	0.88	0.88	1000
8	0.93	0.93	0.93	1000
9	0.90	0.94	0.92	1000
accuracy			0.81	10000
Macro avg.	0.80	0.81	0.80	10000
Weighted avg.	0.80	0.81	0.80	10000

Table 20 Classification Report for the Fashion MNIST dataset for the best model in the test set.

3.2. SVM with/without Kernels

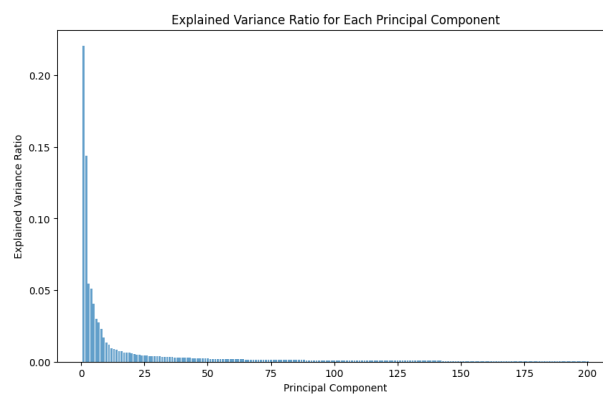


Figure 21 Explained Variance Ratio for Each Principal Component (50) Fashion MNIST for Logistic Regression.

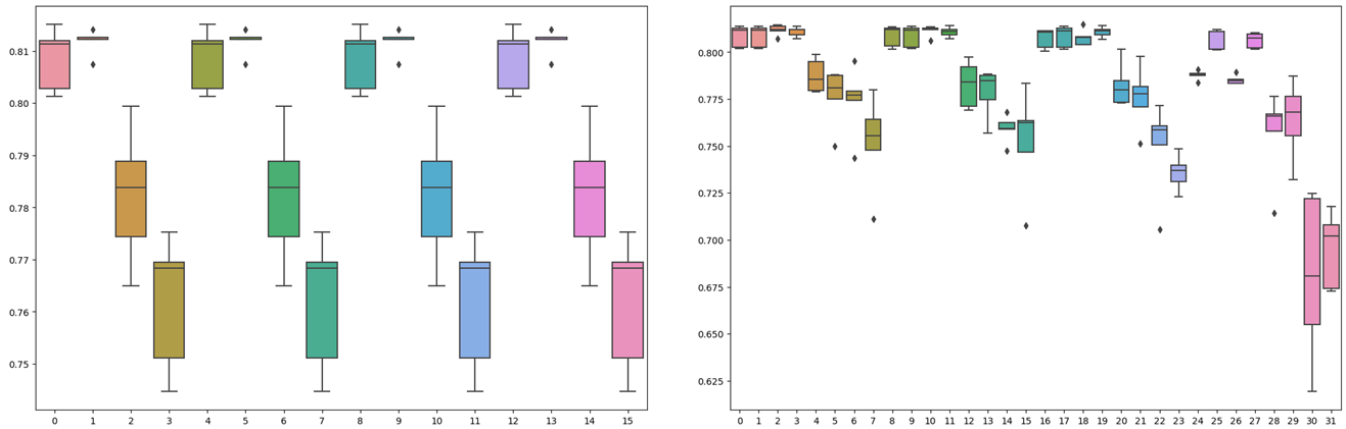


Figure 22 Box plot for the SVM with and without using kernel for 16 and 32 models under the Fashion MNIST Dataset

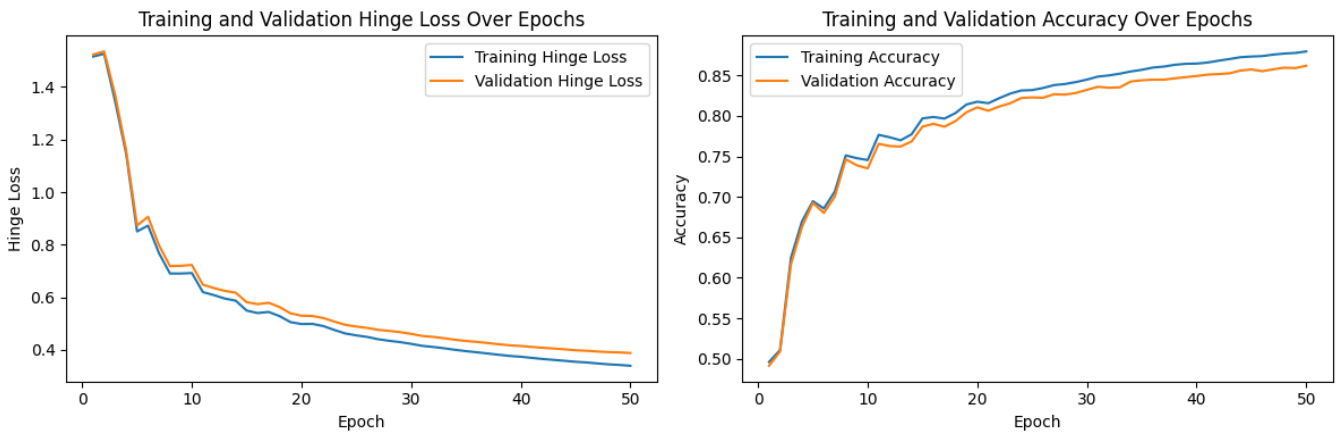


Figure 23 Training and Validation Hinge Loss Over 80 Epochs and Training and Validation Accuracy Over 80 Epochs for the Logistic Regression with regularization of the best model under the Fashion MNIST Dataset.

3.3. PCA + k-NN

The same model application from the UCI Adult Dataset was used for the Wisconsin Breast Cancer dataset for PCA+kNN.

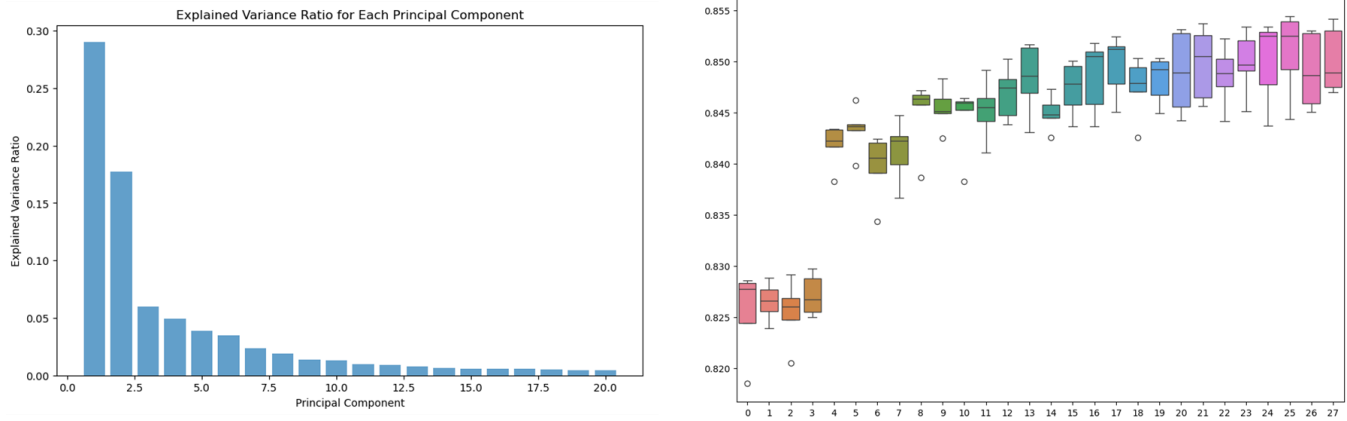


Figure 23 Explained Variance Ratio for Each Principal Component (20) Fashion MNIST Dataset for PCA+kNN and Box plot for PCA+kNN for 28 models under the Fashion MNIST Dataset.

	precision	recall	f1-score	support
0	0.78	0.85	0.81	1000
1	0.98	0.96	0.97	1000
2	0.74	0.79	0.76	1000
3	0.87	0.88	0.87	1000
4	0.75	0.76	0.75	1000
5	0.96	0.89	0.92	1000
6	0.66	0.55	0.60	1000
7	0.88	0.94	0.91	1000
8	0.95	0.95	0.95	1000
9	0.92	0.95	0.94	1000
accuracy			0.85	10000
Macro avg.	0.85	0.85	0.85	10000
Weighted avg.	0.85	0.85	0.85	10000

Table 22 Classification Report for the Fashion MNIST dataset for the best model in the test set.

3.4. FNN

The given Python script is dedicated to a classification task utilizing a PyTorch-based feedforward neural network (FNN) applied to the Fashion MNIST dataset. It begins by loading and processing the dataset, offering a visual representation of a subset of images along with their corresponding class

labels. Subsequently, the script transitions to a machine learning approach using scikit-learn's Support Vector Classifier (SVC) with principal component analysis (PCA) for dimensionality reduction. The code then shifts back to the PyTorch framework, implementing an FNN for the same classification task. A grid search, facilitated by GridSearchCV, is conducted to optimize hyperparameters, and the outcomes are presented through box plots visualizing cross-validation results. The selection of the best model is based on accuracy scores, and its performance is assessed on the test set. Throughout the training process, the script monitors loss and accuracy across epochs, presenting the outcomes in a pair of plots. Finally, the code furnishes key metrics such as testing accuracy and a comprehensive classification report. In summary, the script showcases a methodical and modular approach to constructing, training, and assessing machine learning models, seamlessly integrating both traditional and deep learning techniques.

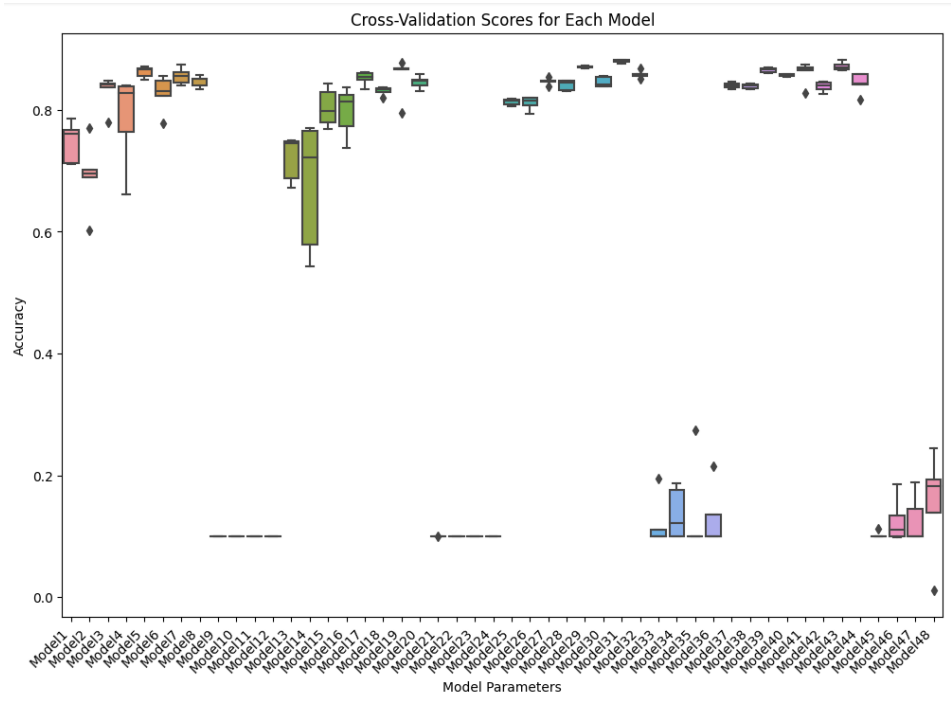


Fig # Boxplots of 48 models of FNN

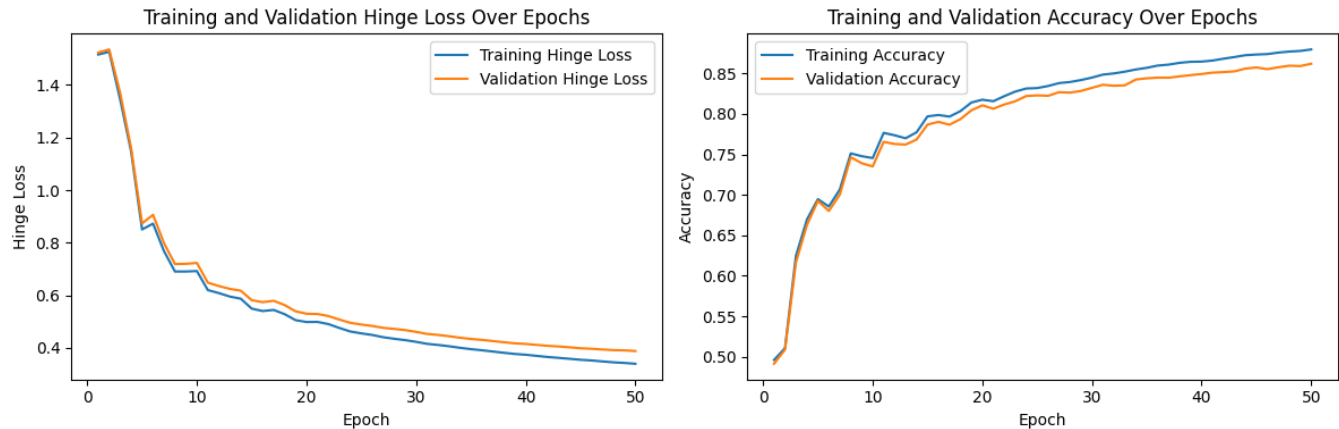


Fig # Training and validation losses and Accuracies for best FNN model

Classification Report:				
	precision	recall	f1-score	support
0	0.80	0.83	0.82	1000
1	0.99	0.96	0.97	1000
2	0.77	0.78	0.77	1000
3	0.85	0.88	0.86	1000
4	0.76	0.79	0.77	1000
5	0.94	0.92	0.93	1000
6	0.71	0.63	0.67	1000
7	0.90	0.94	0.92	1000
8	0.96	0.96	0.96	1000
9	0.94	0.94	0.94	1000
accuracy			0.86	10000
macro avg	0.86	0.86	0.86	10000
weighted avg	0.86	0.86	0.86	10000

Fig # Classification report

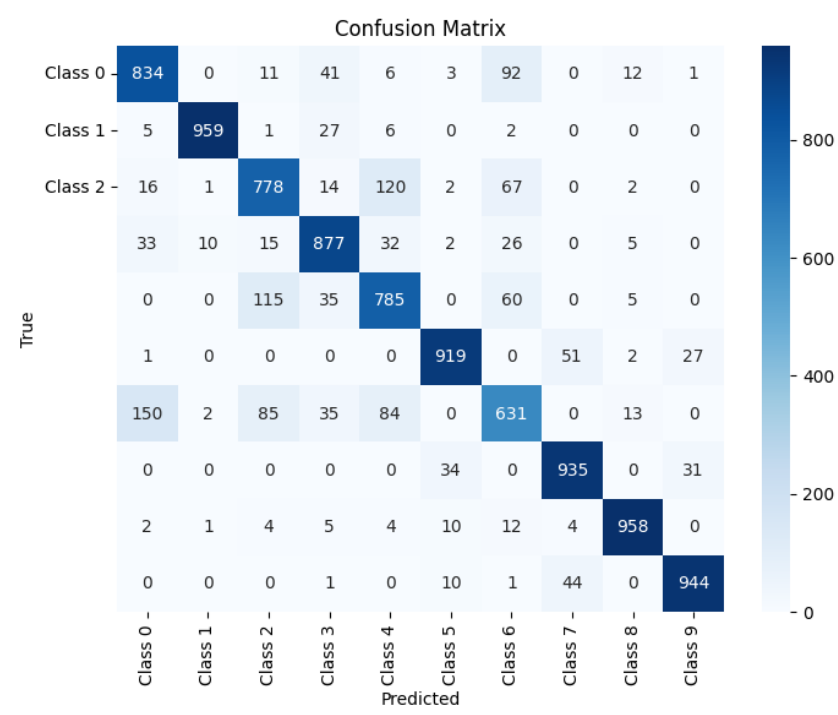


Fig # Confusion matrix

3.5 CNN

The provided Python script begins by loading and preprocessing the Fashion MNIST dataset. The images are flattened, and their mean and standard deviation are calculated. The data is then normalized, and the mean and standard deviation of the normalized data are printed. The script continues by reshaping the data to meet the requirements of a convolutional neural network (CNN) in PyTorch. It also selectively samples 1000 images per class for training, visualizing a couple of images. A CNN class is defined in PyTorch, incorporating convolutional, batch normalization, activation, and pooling layers. The script then defines a PyTorch estimator class, allowing for compatibility with scikit-learn's GridSearchCV. A parameter grid is established for hyperparameter tuning. The script proceeds to conduct a grid search to find the best hyperparameters for the CNN model using cross-validation. The results are visualized using box plots. After saving the best hyperparameters, the script trains the CNN model on the training set, monitoring loss and accuracy over epochs. The results are plotted for analysis. Finally, the script evaluates the trained model on the test set, calculating testing accuracy and providing a classification report.

Code

1. UCI Adult Dataset

1.1. Logistic Regression with Regularization

The code can be run all together. There are no specific instructions for the code.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from sklearn.linear_model import SGDClassifier
from sklearn.kernel_approximation import RBFSampler
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score, classification_report, roc_curve
from sklearn.metrics import roc_auc_score, log_loss, confusion_matrix
import seaborn as sns
import pandas as pd
from sklearn.model_selection import train_test_split

columns = [
    "age", "workclass", "fnlwgt", "education", "education-num", "marital-status",
    "occupation", "relationship", "race", "sex", "capital-gain", "capital-loss",
    "hours-per-week", "native-country", "income"
]
```

```

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data"
df = pd.read_csv(url, header=None, names=columns, na_values=" ?", skipinitialspace=True)
df.replace("?", pd.NA, inplace=True)
df = df.dropna()
categorical_columns = ["workclass", "education", "marital-status", "occupation", "relationship", "race",
"sex", "native-country"]
df = pd.get_dummies(df, columns=categorical_columns)

# print(df.head())

column_to_move = df.columns[6] # Assuming 0-based indexing
df = df.assign(**{column_to_move: df.pop(column_to_move)})

y = df.iloc[:, -1]

columns_to_standardize = ['age', 'fnlwgt', 'education-num', 'capital-gain', 'capital-loss', 'hours-per-week']
X_to_standardize = df[columns_to_standardize]

scaler = StandardScaler()
X_standardized = pd.DataFrame(scaler.fit_transform(X_to_standardize),
columns=columns_to_standardize)

X = pd.concat([X_standardized, df.drop(columns=columns_to_standardize).reset_index(drop=True)],
axis=1)

X = X.iloc[:, :-1]

y_new = []
for val in y.values:
    if val[0] == "<":
        y_new.append(0)
    else:
        y_new.append(1)

y = np.array(y_new)
X = X.values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define the model
model = SGDClassifier(random_state=42)

# Define the hyperparameter grid
param_grid = {
    'alpha': [0.00001, 0.0001, 0.001, 0.01, .1],

```

```

'penalty': ['l1', 'l2'],
'max_iter': [1000],
'loss' : ['log_loss'],
'learning_rate' : ['constant'],
'eta0' : [0.00001, 0.0001, 0.001, 0.01, .1]
}

```

```

# Create GridSearchCV object with cross-validation
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)

```

```

# Get the best parameters and the corresponding model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

```

```

all_models = grid_search.cv_results_['params']

```

```

# Use cross_val_score to get cross-validation scores for each model
cross_val_scores = []
for model_params in all_models:
    model = SGDClassifier(**model_params, random_state=42)
    scores = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy')
    cross_val_scores.append(scores)

```

```

plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)
plt.xticks(ticks=range(len(all_models)))

```

```

# Simulate partial_fit with batch-wise training
batch_size = X_train.shape[0]
num_samples = X_train.shape[0]
num_batches = int(np.ceil(num_samples / batch_size))
print("Best model : ", best_params)

```

```

# Training and validation losses
epochs = [20, 80]
for each in epochs:
    train_losses = []
    val_losses = []
    train_accuracies = []
    val_accuracies = []

```

```

# best_sgd_model = SGDClassifier(**best_params)

```

```

best_sgd_model = SGDClassifier(alpha=best_params['alpha'], eta0=best_params['eta0'],
                               learning_rate=best_params['learning_rate'], loss=best_params['loss'],
                               max_iter=best_params['max_iter'], penalty=best_params['penalty'], random_state=42)

# Learning rate schedule
initial_learning_rate = 0.01
learning_rate = initial_learning_rate

# Early stopping parameters
patience = 5
best_val_loss = float('inf')
no_improvement_count = 0

for epoch in range(each):
    for batch_start in range(0, num_samples, batch_size):
        batch_end = min(batch_start + batch_size, num_samples)
        X_batch = X_train[batch_start:batch_end]
        y_batch = y_train[batch_start:batch_end]

        # Train the model on the batch
        best_sgd_model.partial_fit(X_batch, y_batch, classes=np.unique(y_train))
    # print(epoch)

    # Evaluate on training and validation sets
    train_predictions = best_sgd_model.predict(X_train)
    val_predictions = best_sgd_model.predict(X_test)
    train_loss = log_loss(y_train, train_predictions)
    val_loss = log_loss(y_test, val_predictions)
    train_losses.append(train_loss)
    val_losses.append(val_loss)
    train_accuracy = accuracy_score(y_train, train_predictions)
    val_accuracy = accuracy_score(y_test, val_predictions)
    train_accuracies.append(train_accuracy)
    val_accuracies.append(val_accuracy)

# Plotting
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(range(1, len(train_losses) + 1), train_losses, label='Training Log Loss')
plt.plot(range(1, len(val_losses) + 1), val_losses, label='Validation Log Loss')
plt.title('Training and Validation Log Loss Over Epochs')
plt.xlabel('Epoch')

```



```
plt.ylabel('Log Loss')
plt.legend()
```

```
plt.subplot(1, 2, 2)
plt.plot(range(1, len(train_accuracies) + 1), train_accuracies, label='Training Accuracy')
plt.plot(range(1, len(val_accuracies) + 1), val_accuracies, label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.tight_layout()
plt.show()
```

```
# Evaluate the best model on the test set
y_pred = best_sgd_model.predict(X_test)
y_prob = best_sgd_model.decision_function(X_test)
```

```
# Testing accuracy
test_accuracy = accuracy_score(y_test, y_pred)
print("Testing Accuracy:", test_accuracy)
```

```
# Classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
# Confusion matrix
confusion_mat = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(confusion_mat)
```

```
# ROC curve and AUC score
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc_score = roc_auc_score(y_test, y_prob)
plt.figure(figsize=(8, 8))
plt.plot(fpr, tpr, label=f'AUC = {auc_score:.2f}')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.title('ROC Curve')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.show()
```

1.2. Logistic Regression without Regularization

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from sklearn.linear_model import SGDClassifier
from sklearn.kernel_approximation import RBFSampler
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score, classification_report, roc_curve
from sklearn.metrics import roc_auc_score, log_loss, confusion_matrix
import seaborn as sns
import pandas as pd
from sklearn.model_selection import train_test_split

columns = [
    "age", "workclass", "fnlwt", "education", "education-num", "marital-status",
    "occupation", "relationship", "race", "sex", "capital-gain", "capital-loss",
    "hours-per-week", "native-country", "income"
]

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data"
df = pd.read_csv(url, header=None, names=columns, na_values="?", skipinitialspace=True)
df.replace("?", pd.NA, inplace=True)
df = df.dropna()
categorical_columns = ["workclass", "education", "marital-status", "occupation", "relationship", "race",
"sex", "native-country"]
df = pd.get_dummies(df, columns=categorical_columns)

# print(df.head())

column_to_move = df.columns[6] # Assuming 0-based indexing
df = df.assign(**{column_to_move: df.pop(column_to_move)})

y = df.iloc[:, -1]

columns_to_standardize = ['age', 'fnlwt', 'education-num', 'capital-gain', 'capital-loss', 'hours-per-week']
X_to_standardize = df[columns_to_standardize]

scaler = StandardScaler()
X_standardized = pd.DataFrame(scaler.fit_transform(X_to_standardize),
columns=columns_to_standardize)

X = pd.concat([X_standardized, df.drop(columns=columns_to_standardize).reset_index(drop=True)],
axis=1)
```

```

X = X.iloc[:, :-1]

y_new = []
for val in y.values:
    if val[0] == "<":
        y_new.append(0)
    else:
        y_new.append(1)

y = np.array(y_new)
X = X.values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define the model
model = SGDClassifier(random_state=42)

# Define the hyperparameter grid
param_grid = {
    'alpha': [0.00001, 0.0001, 0.001, 0.01, .1],
    'penalty': [None],
    'max_iter': [1000],
    'loss' : ['log_loss'],
    'learning_rate' : ['constant'],
    'eta0' : [0.00001, 0.0001, 0.001, 0.01, .1]
}

# Create GridSearchCV object with cross-validation
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)

# Get the best parameters and the corresponding model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

all_models = grid_search.cv_results_['params']

# Use cross_val_score to get cross-validation scores for each model
cross_val_scores = []
for model_params in all_models:
    model = SGDClassifier(**model_params, random_state=42)
    scores = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy')
    cross_val_scores.append(scores)

```

```
plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)
plt.xticks(ticks=range(len(all_models)))
```

```
# Simulate partial_fit with batch-wise training
batch_size = X_train.shape[0]
num_samples = X_train.shape[0]
num_batches = int(np.ceil(num_samples / batch_size))
print("Best model : ", best_params)
```

```
# Training and validation losses
```

```
epochs = [20, 80]
```

```
for each in epochs:
```

```
    train_losses = []
```

```
    val_losses = []
```

```
    train_accuracies = []
```

```
    val_accuracies = []
```

```
# best_sgd_model = SGDClassifier(**best_params)
```

```
best_sgd_model = SGDClassifier(alpha=best_params['alpha'], eta0=best_params['eta0'],
                               learning_rate=best_params['learning_rate'], loss=best_params['loss'],
```

```
max_iter=best_params['max_iter'], penalty=best_params['penalty'], random_state=42)
```

```
# Learning rate schedule
```

```
initial_learning_rate = 0.01
```

```
learning_rate = initial_learning_rate
```

```
# Early stopping parameters
```

```
patience = 5
```

```
best_val_loss = float('inf')
```

```
no_improvement_count = 0
```

```
for epoch in range(each):
```

```
    for batch_start in range(0, num_samples, batch_size):
```

```
        batch_end = min(batch_start + batch_size, num_samples)
```

```
        X_batch = X_train[batch_start:batch_end]
```

```
        y_batch = y_train[batch_start:batch_end]
```

```
        # Train the model on the batch
```

```
        best_sgd_model.partial_fit(X_batch, y_batch, classes=np.unique(y_train))
```

```
    # print(epoch)
```

```

# Evaluate on training and validation sets
train_predictions = best_sgd_model.predict(X_train)
val_predictions = best_sgd_model.predict(X_test)
train_loss = log_loss(y_train, train_predictions)
val_loss = log_loss(y_test, val_predictions)
train_losses.append(train_loss)
val_losses.append(val_loss)
train_accuracy = accuracy_score(y_train, train_predictions)
val_accuracy = accuracy_score(y_test, val_predictions)
train_accuracies.append(train_accuracy)
val_accuracies.append(val_accuracy)

# Plotting
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(range(1, len(train_losses) + 1), train_losses, label='Training Log Loss')
plt.plot(range(1, len(val_losses) + 1), val_losses, label='Validation Log Loss')
plt.title('Training and Validation Log Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Log Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(1, len(train_accuracies) + 1), train_accuracies, label='Training Accuracy')
plt.plot(range(1, len(val_accuracies) + 1), val_accuracies, label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.tight_layout()
plt.show()

# Evaluate the best model on the test set
y_pred = best_sgd_model.predict(X_test)
y_prob = best_sgd_model.decision_function(X_test)

# Testing accuracy
test_accuracy = accuracy_score(y_test, y_pred)
print("Testing Accuracy:", test_accuracy)

# Classification report
print("Classification Report:")

```

```

print(classification_report(y_test, y_pred))

# Confusion matrix
confusion_mat = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(confusion_mat)

# ROC curve and AUC score
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc_score = roc_auc_score(y_test, y_prob)
plt.figure(figsize=(8, 8))
plt.plot(fpr, tpr, label=f'AUC = {auc_score:.2f}')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.title('ROC Curve')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.show()

```

1.3. SVM with Kernels

```

import pandas as pd

columns = [
    "age", "workclass", "fnlwgt", "education", "education-num", "marital-status",
    "occupation", "relationship", "race", "sex", "capital-gain", "capital-loss",
    "hours-per-week", "native-country", "income"
]

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data"
df = pd.read_csv(url, header=None, names=columns, na_values=" ?", skipinitialspace=True)
df.replace("?", pd.NA, inplace=True)
df = df.dropna()
categorical_columns = ["workclass", "education", "marital-status", "occupation", "relationship", "race",
"sex", "native-country"]
df = pd.get_dummies(df, columns=categorical_columns)

# print(df.head())

import pandas as pd

column_to_move = df.columns[6] # Assuming 0-based indexing
df = df.assign(**{column_to_move: df.pop(column_to_move)})

```

df

```
from sklearn.preprocessing import StandardScaler
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
```

```
y = df.iloc[:, -1]
```

```
columns_to_standardize = ['age', 'fnlwgt', 'education-num', 'capital-gain', 'capital-loss', 'hours-per-week']
X_to_standardize = df[columns_to_standardize]
```

```
scaler = StandardScaler()
X_standardized = pd.DataFrame(scaler.fit_transform(X_to_standardize),
                               columns=columns_to_standardize)
```

```
X = pd.concat([X_standardized, df.drop(columns=columns_to_standardize).reset_index(drop=True)],
               axis=1)
```

```
X = X.iloc[:, :-1]
```

```
y_new = []
for val in y.values:
    if val[0] == "<":
        y_new.append(0)
    else:
        y_new.append(1)
```

```
y = np.array(y_new)
X = X.values
```

```
# Apply PCA
pca = PCA(n_components=11, random_state=40)
X_pca = pca.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(X_pca, y, test_size=0.2, random_state=42)
```

```
import matplotlib.pyplot as plt
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from sklearn.linear_model import SGDClassifier
from sklearn.kernel_approximation import RBFSampler
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris
```

```

from sklearn.metrics import accuracy_score, classification_report, roc_curve, roc_auc_score,
hinge_loss
from sklearn.metrics import log_loss

import numpy as np

# Define the RBF Sampler
rbf_sampler = RBFSampler(gamma=0.01, random_state=42)

# Apply kernel approximation to the training and testing data
X_train_transformed = rbf_sampler.fit_transform(X_train)
X_test_transformed = rbf_sampler.transform(X_test)

# Define the model
model = SGDClassifier(random_state=42, )

# Define the hyperparameter grid
param_grid = {
    'alpha': [0.0001, 0.001],
    'penalty': ['l1', 'l2'],
    'max_iter': [1000],
    'loss' : ['hinge'],
    'learning_rate' : ['constant'],
    'eta0' : [0.0001, 0.001, 0.01]
}

# Create GridSearchCV object with cross-validation
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train_transformed, y_train)

# Get the best parameters and the corresponding model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split

all_models = grid_search.cv_results_['params']

# Use cross_val_score to get cross-validation scores for each model
cross_val_scores = []
for model_params in all_models:
    model = SGDClassifier(**model_params)

```



```
scores = cross_val_score(model, X_train_transformed, y_train, cv=5, scoring='accuracy') # Fix the
variable name
cross_val_scores.append(scores)
```

```
plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)
plt.xticks(ticks=range(len(all_models)), labels=["Model" + str(i+1) for i in range(len(all_models))],
rotation=45, ha='right')
plt.title('Cross-Validation Scores for Each Model')
plt.xlabel('Model Parameters')
plt.ylabel('Accuracy')
plt.show()
```

```
print('Cross validation scores : ', cross_val_scores)
```

```
# Simulate partial_fit with batch-wise training
batch_size = 32
num_samples = X_train_transformed.shape[0]
num_batches = int(np.ceil(num_samples / batch_size))
```

```
print("Best model : ", best_params)
```

```
# Training and validation losses
```

```
epochs = 60
train_losses = []
val_losses = []
train_accuracies = []
val_accuracies = []
```

```
for epoch in range(epochs):
    for batch_start in range(0, num_samples, batch_size):
        batch_end = min(batch_start + batch_size, num_samples)
        X_batch = X_train_transformed[batch_start:batch_end]
        y_batch = y_train[batch_start:batch_end]
```

```
    # Train the model on the batch
    best_model.partial_fit(X_batch, y_batch, classes=np.unique(y_train))
```

```
# Evaluate on training and validation sets
```

```
train_predictions = best_model.predict(X_train_transformed)
val_predictions = best_model.predict(X_test_transformed)
```

```
train_loss = hinge_loss(y_train, best_model.decision_function(X_train_transformed))
val_loss = hinge_loss(y_test, best_model.decision_function(X_test_transformed))
```

```
train_losses.append(train_loss)
val_losses.append(val_loss)
```

```
train_accuracy = accuracy_score(y_train, train_predictions)
val_accuracy = accuracy_score(y_test, val_predictions)
```

```
train_accuracies.append(train_accuracy)
val_accuracies.append(val_accuracy)
```

```
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(range(1, epochs + 1), train_losses, label='Training Hinge Loss')
plt.plot(range(1, epochs + 1), val_losses, label='Validation Hinge Loss')
plt.title('Training and Validation Hinge Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Hinge Loss')
plt.legend()
```

```
plt.subplot(1, 2, 2)
plt.plot(range(1, epochs + 1), train_accuracies, label='Training Accuracy')
plt.plot(range(1, epochs + 1), val_accuracies, label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
```

```
plt.tight_layout()
plt.show()
```

```
# Evaluate the best model on the test set
y_pred = best_model.predict(X_test_transformed)
y_prob = best_model.decision_function(X_test_transformed)
```

```
# Testing accuracy
test_accuracy = accuracy_score(y_test, y_pred)
print("Testing Accuracy:", test_accuracy)
```

```
# Classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

1.4. SVM without Kernels

```
import pandas as pd

columns = [
    "age", "workclass", "fnlwgt", "education", "education-num", "marital-status",
    "occupation", "relationship", "race", "sex", "capital-gain", "capital-loss",
    "hours-per-week", "native-country", "income"
]

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data"
df = pd.read_csv(url, header=None, names=columns, na_values="?", skipinitialspace=True)
df.replace("?", pd.NA, inplace=True)
df = df.dropna()
categorical_columns = ["workclass", "education", "marital-status", "occupation", "relationship", "race",
"sex", "native-country"]
df = pd.get_dummies(df, columns=categorical_columns)

# print(df.head())
import pandas as pd

column_to_move = df.columns[6] # Assuming 0-based indexing
df = df.assign(**{column_to_move: df.pop(column_to_move)})

df
from sklearn.preprocessing import StandardScaler
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA

y = df.iloc[:, -1]

columns_to_standardize = ['age', 'fnlwgt', 'education-num', 'capital-gain', 'capital-loss', 'hours-per-week']
X_to_standardize = df[columns_to_standardize]

scaler = StandardScaler()
X_standardized = pd.DataFrame(scaler.fit_transform(X_to_standardize),
columns=columns_to_standardize)

X = pd.concat([X_standardized, df.drop(columns=columns_to_standardize).reset_index(drop=True)],
axis=1)

X = X.iloc[:, :-1]

y_new = []
```

```

for val in y.values:
    if val[0] == "<":
        y_new.append(0)
    else:
        y_new.append(1)

y = np.array(y_new)
X = X.values

# Apply PCA
pca = PCA(n_components=11, random_state=40)
X_pca = pca.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_pca, y, test_size=0.2, random_state=42)

import matplotlib.pyplot as plt
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from sklearn.linear_model import SGDClassifier
from sklearn.kernel_approximation import RBFSampler
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score, classification_report, roc_curve, roc_auc_score,
hinge_loss
from sklearn.metrics import log_loss

import numpy as np

## Apply kernel approximation to the training and testing data
X_train_transformed = (X_train)
X_test_transformed = (X_test)

# Define the model
model = SGDClassifier(random_state=42, )

# Define the hyperparameter grid
param_grid = {
    'alpha': [0.0001, 0.001],
    'penalty': ['l1', 'l2'],
    'max_iter': [1000],
    'loss' : ['hinge'],
    'learning_rate' : ['constant'],
    'eta0' : [0.0001, 0.001, 0.01]
}

```

```

# Simulate partial_fit with batch-wise training
batch_size = 32
num_samples = X_train_transformed.shape[0] # Define num_samples based on the shape of
X_train_transformed
num_batches = int(np.ceil(num_samples / batch_size))

# Training and validation losses
epochs = 60
train_losses = []
val_losses = []
train_accuracies = []
val_accuracies = []

for epoch in range(epochs):
    for batch_start in range(0, num_samples, batch_size):
        batch_end = min(batch_start + batch_size, num_samples)
        X_batch = X_train_transformed[batch_start:batch_end]
        y_batch = y_train[batch_start:batch_end]

        # Train the model on the batch
        best_model.partial_fit(X_batch, y_batch, classes=np.unique(y_train))

    # Evaluate on training and validation sets
    train_predictions = best_model.predict(X_train_transformed)
    val_predictions = best_model.predict(X_test_transformed)

    # Use hinge_loss correctly for multiclass classification
    train_loss = hinge_loss(y_train, best_model.decision_function(X_train_transformed),
labels=np.unique(y_train))
    val_loss = hinge_loss(y_test, best_model.decision_function(X_test_transformed),
labels=np.unique(y_test))

    train_losses.append(train_loss)
    val_losses.append(val_loss)

    train_accuracy = accuracy_score(y_train, train_predictions)
    val_accuracy = accuracy_score(y_test, val_predictions)

    train_accuracies.append(train_accuracy)
    val_accuracies.append(val_accuracy)

# Plotting code moved outside the loop
plt.figure(figsize=(12, 4))

```

```
plt.subplot(1, 2, 1)
plt.plot(range(1, epochs + 1), train_losses, label='Training Hinge Loss')
plt.plot(range(1, epochs + 1), val_losses, label='Validation Hinge Loss')
plt.title('Training and Validation Hinge Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Hinge Loss')
plt.legend()
```

```
plt.subplot(1, 2, 2)
plt.plot(range(1, epochs + 1), train_accuracies, label='Training Accuracy')
plt.plot(range(1, epochs + 1), val_accuracies, label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
```

```
plt.tight_layout()
plt.show()
```

```
# Create GridSearchCV object with cross-validation
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train_transformed, y_train)
```

```
# Get the best parameters and the corresponding model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_
```

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
```

```
all_models = grid_search.cv_results_['params']
```

```
# Use cross_val_score to get cross-validation scores for each model
cross_val_scores = []
for model_params in all_models:
    model = SGDClassifier(**model_params)
    scores = cross_val_score(model, X_train_transformed, y_train, cv=5, scoring='accuracy') # Fix the
    variable name
    cross_val_scores.append(scores)
```

```
plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)
```

```
plt.xticks(ticks=range(len(all_models)), labels=['Model' + str(i+1) for i in range(len(all_models))],
rotation=45, ha='right')
plt.title('Cross-Validation Scores for Each Model')
plt.xlabel('Model Parameters')
plt.ylabel('Accuracy')
plt.show()
```

```
print('Cross validation scores : ', cross_val_scores)
```

```
# Plot training and validation losses
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(range(1, epochs + 1), train_losses, label='Training Hinge Loss')
plt.plot(range(1, epochs + 1), val_losses, label='Validation Hinge Loss')
plt.title('Training and Validation Hinge Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Hinge Loss')
plt.legend()
```

```
# Plot training and validation accuracies
plt.subplot(1, 2, 2)
plt.plot(range(1, epochs + 1), train_accuracies, label='Training Accuracy')
plt.plot(range(1, epochs + 1), val_accuracies, label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
```

```
plt.tight_layout()
plt.show()
```

```
# Simulate partial_fit with batch-wise training
batch_size = 32
num_samples = X_train_transformed.shape[0]
num_batches = int(np.ceil(num_samples / batch_size))
```

```
print("Best model : ", best_params)
```

```
# Training and validation losses
epochs = 60
train_losses = []
val_losses = []
train_accuracies = []
val_accuracies = []
```

```

for epoch in range(epochs):
    for batch_start in range(0, num_samples, batch_size):
        batch_end = min(batch_start + batch_size, num_samples)
        X_batch = X_train_transformed[batch_start:batch_end]
        y_batch = y_train[batch_start:batch_end]

        # Train the model on the batch
        best_model.partial_fit(X_batch, y_batch, classes=np.unique(y_train))

    # Evaluate on training and validation sets
    train_predictions = best_model.predict(X_train_transformed)
    val_predictions = best_model.predict(X_test_transformed)

    train_loss = hinge_loss(y_train, best_model.decision_function(X_train_transformed))
    val_loss = hinge_loss(y_test, best_model.decision_function(X_test_transformed))

    train_losses.append(train_loss)
    val_losses.append(val_loss)

    train_accuracy = accuracy_score(y_train, train_predictions)
    val_accuracy = accuracy_score(y_test, val_predictions)

    train_accuracies.append(train_accuracy)
    val_accuracies.append(val_accuracy)

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(range(1, epochs + 1), train_losses, label='Training Hinge Loss')
plt.plot(range(1, epochs + 1), val_losses, label='Validation Hinge Loss')
plt.title('Training and Validation Hinge Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Hinge Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(1, epochs + 1), train_accuracies, label='Training Accuracy')
plt.plot(range(1, epochs + 1), val_accuracies, label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.tight_layout()

```



```

plt.show()

# Evaluate the best model on the test set
y_pred = best_model.predict(X_test_transformed)
y_prob = best_model.decision_function(X_test_transformed)

# Testing accuracy
test_accuracy = accuracy_score(y_test, y_pred)
print("Testing Accuracy:", test_accuracy)

# Classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))

```

1.5. PCA + k-NN

```

import numpy as np
from ucimlrepo import fetch_ucirepo
import matplotlib.pyplot as plt
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from sklearn.linear_model import SGDClassifier
from sklearn.kernel_approximation import RBFSampler
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score, classification_report, roc_curve
from sklearn.metrics import roc_auc_score, log_loss, confusion_matrix
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
import seaborn as sns
import pandas as pd

columns = [
    "age", "workclass", "fnlwgt", "education", "education-num", "marital-status",
    "occupation", "relationship", "race", "sex", "capital-gain", "capital-loss",
    "hours-per-week", "native-country", "income"
]

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data"
df = pd.read_csv(url, header=None, names=columns, na_values="?", skipinitialspace=True)
df.replace("?", pd.NA, inplace=True)
df = df.dropna()
categorical_columns = ["workclass", "education", "marital-status", "occupation", "relationship", "race",
"sex", "native-country"]
df = pd.get_dummies(df, columns=categorical_columns)

```

```

# print(df.head())

column_to_move = df.columns[6] # Assuming 0-based indexing
df = df.assign(**{column_to_move: df.pop(column_to_move)})

y = df.iloc[:, -1]

columns_to_standardize = ['age', 'fnlwt', 'education-num', 'capital-gain', 'capital-loss', 'hours-per-week']
X_to_standardize = df[columns_to_standardize]

scaler = StandardScaler()
X_standardized = pd.DataFrame(scaler.fit_transform(X_to_standardize),
                               columns=columns_to_standardize)

X = pd.concat([X_standardized, df.drop(columns=columns_to_standardize).reset_index(drop=True)],
              axis=1)

X = X.iloc[:, :-1]

y_new = []
for val in y.values:
    if val[0] == "<":
        y_new.append(0)
    else:
        y_new.append(1)

y = np.array(y_new)
X = X.values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

max_components = min(X_test.shape[0], X_test.shape[1])
print("Maximum number of components:", max_components)

num_components= [5,50,75]
for every in num_components:
    # Apply PCA for dimensionality reduction (optional)
    # You can skip this step or adjust the number of components based on your needs
    # num_components = 15 # You can choose the number of components you want
    pca = PCA(n_components=every)
    # Fit and transform the training data
    X_train_transformed = pca.fit_transform(X_train)
    # Transform the test data
    X_test_transformed = pca.transform(X_test)

```

```

# Visualize the first two principal components
explained_var_ratio = pca.explained_variance_ratio_
# Scree plot
plt.figure(figsize=(10, 6))
plt.bar(range(1, every + 1), explained_var_ratio, alpha=0.7, align='center')
plt.title('Explained Variance Ratio for Each Principal Component')
plt.xlabel('Principal Component')
plt.ylabel('Explained Variance Ratio')
plt.show()

# Define the model
model = KNeighborsClassifier()

# Define the hyperparameter grid
param_grid = {
    'n_neighbors': [3,5,7,9],
    'weights': ['uniform','distance'],
    'p': [1,2]
}

# Create GridSearchCV object with cross-validation
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train_transformed, y_train)

# Get the best parameters and the corresponding model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

all_models = grid_search.cv_results_['params']

# Use cross_val_score to get cross-validation scores for each model
cross_val_scores = []
for model_params in all_models:
    model = KNeighborsClassifier(**model_params)
    scores = cross_val_score(model, X_train_transformed, y_train, cv=5, scoring='accuracy')
    cross_val_scores.append(scores)

plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)
plt.xticks(ticks=range(len(all_models)))

# Simulate partial_fit with batch-wise training
batch_size = X_train.shape[0]
num_samples = X_train.shape[0]

```

```

num_batches = int(np.ceil(num_samples / batch_size))
print("Best model : ", best_params)

# best_sgd_model = KNeighborsClassifier(**best_params)
best_knn_model = KNeighborsClassifier(n_neighbors=best_params['n_neighbors'])
# Train the model
best_knn_model.fit(X_train_transformed, y_train)

# Evaluate the best model on the test set
y_pred = best_knn_model.predict(X_test_transformed)
y_prob = best_knn_model.predict_proba(X_test_transformed)

# Testing accuracy
test_accuracy = accuracy_score(y_test, y_pred)
print("Testing Accuracy:", test_accuracy)

# Classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))

# Confusion matrix
confusion_mat = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(confusion_mat)

```

1.6. FNN

```

!pip install ucimlrepo

from ucimlrepo import fetch_ucirepo

breast_cancer_wisconsin_diagnostic = fetch_ucirepo(id=17)

X = breast_cancer_wisconsin_diagnostic.data.features
y = breast_cancer_wisconsin_diagnostic.data.targets

from sklearn.preprocessing import StandardScaler
import numpy as np
from sklearn.model_selection import train_test_split

scaler = StandardScaler()

```

```

X = scaler.fit_transform(X)

class_mapping = {'M': 1, 'B': 0}

y = [class_mapping[target] for target in y.T.values[0]]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score

input_size = 30
output_size = 1
num_epochs = 50
batch_size = 32

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

X_train = torch.tensor(X_train,
dtype=torch.float32).clone().detach().to(device).type(torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).clone().detach().view(-1,
1).to(device).type(torch.float32)
X_test = torch.tensor(X_test,
dtype=torch.float32).clone().detach().to(device).type(torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32).clone().detach().view(-1,
1).to(device).type(torch.float32)

class FNN(nn.Module):
    def __init__(self, input_size, hidden_sizes, output_size, dropout_prob=0.4):
        super(FNN, self).__init__()
        layer_sizes = [input_size] + hidden_sizes + [output_size]
        layers = []
        for i in range(len(layer_sizes) - 1):
            layers.append(nn.Linear(layer_sizes[i], layer_sizes[i + 1]))
            if i < len(layer_sizes) - 1:
                layers.append(nn.ReLU()) # Adding ReLU activation between hidden layers
            if i < len(layer_sizes) - 2:
                layers.append(nn.Dropout(dropout_prob)) # Dropout layer between hidden
layers

```

```
layers.append(nn.Sigmoid()) # Sigmoid activation for the output layer
self.model = nn.Sequential(*layers)
```

```
def forward(self, x):
    return self.model(x)
```

```
from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
```

```
param_grid = {
    'lr': [0.01, 0.001],
    'hidden_sizes': [[32, 16], [64, 16], [64, 8]],
    'input_size': [X_train.shape[1]],
    'output_size': [1],
    'criterion': [nn.BCELoss()],
    'optimizer_class': [optim.Adam],
    'wd': [0.001, 0.01],
    'num_epochs': [50],
    'batch_size': [32, X_train.shape[0]],
    'model_class' : [FNN]
}
```

```
class PyTorchEstimator(BaseEstimator, ClassifierMixin):
    def __init__(self, model_class, criterion, optimizer_class, input_size=30, hidden_sizes
= [32, 16], output_size = 1, lr=0.001, wd=0.0, num_epochs=20, batch_size=32, model =
None):
        self.model_class = model_class
        self.criterion = criterion
        self.optimizer_class = optimizer_class
        self.input_size = input_size
        self.hidden_sizes = hidden_sizes
        self.output_size = output_size
        self.lr = lr
        self.wd = wd
        self.num_epochs = num_epochs
        self.batch_size = batch_size
        self.model = None
```

```
def fit(self, X, y):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    X_tensor = torch.tensor(X, dtype=torch.float32).to(device)
    y_tensor = torch.tensor(y, dtype=torch.float32).view(-1, 1).to(device)
```

```

num_samples = X.shape[0]

self.model = self.model_class(self.input_size, self.hidden_sizes,
self.output_size).to(device)
optimizer = self.optimizer_class(self.model.parameters(), lr=self.lr,
weight_decay=self.wd)

for epoch in range(self.num_epochs):
    self.model.train()
    for batch_start in range(0, num_samples, self.batch_size):
        batch_end = min(batch_start + self.batch_size, num_samples)
        X_batch = X_tensor[batch_start:batch_end]
        y_batch = y_tensor[batch_start:batch_end]

        optimizer.zero_grad()
        y_pred = self.model(X_batch)

        loss = self.criterion(y_pred, y_batch)
        loss.backward()

        optimizer.step()

    if epoch % 10 == 0:
        print(f'Epoch {epoch + 1} : Parameters - Batch size: {self.batch_size}, lr:
{self.lr}, wd: {self.wd}')

    return self

def predict(self, X):
    self.model.eval()
    with torch.no_grad():
        X_tensor = torch.tensor(X, dtype=torch.float32).to(device)
        y_pred = self.model(X_tensor)
        y_pred_binary = (y_pred > 0.5).float()
    return y_pred_binary.cpu().numpy()

def get_params(self, deep=True):
    return {
        'model_class': self.model_class,
        'criterion': self.criterion,
        'optimizer_class': self.optimizer_class,
        'input_size': self.input_size,
        'hidden_sizes': self.hidden_sizes,

```

```

        'output_size': self.output_size,
        'lr': self.lr,
        'num_epochs': self.num_epochs,
        'batch_size': self.batch_size,
        'model': self.model,
        'wd': self.wd
    }

```

```

pytorch_estimator = PyTorchEstimator(FNN, nn.BCELoss(), optim.Adam)
#print(X_train.device, y_train.device)
grid_search = GridSearchCV(pytorch_estimator, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train.cpu(), y_train.cpu())

```

```

best_params = grid_search.best_params_

```

```

import seaborn as sns
import matplotlib.pyplot as plt

```

```

all_models = grid_search.cv_results_['params']

```

```

cross_val_scores = []
for model_params in all_models:
    model = PyTorchEstimator(**model_params)
    scores = cross_val_score(model, X_train.cpu(), y_train.cpu(), cv=5,
scoring='accuracy')
    cross_val_scores.append(scores)

```

```

plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)
plt.xticks(ticks=range(len(all_models)), labels=['Model' + str(i+1) for i in
range(len(all_models))], rotation=45, ha='right')
plt.title('Cross-Validation Scores for Each Model')
plt.xlabel('Model Parameters')
plt.ylabel('Accuracy')
plt.show()
print("Best model : ", best_params)

```

```

import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, classification_report, roc_curve,
roc_auc_score
import matplotlib.pyplot as plt

```



```

num_samples = X_train.shape[0]
num_batches = int(np.ceil(num_samples / num_samples))

model = best_params['model_class'](best_params['input_size'],
best_params['hidden_sizes'], best_params['output_size']).to(device)
optimizer = best_params['optimizer_class'](model.parameters(), lr=best_params['lr'],
weight_decay = best_params['wd'])

train_losses = []
val_losses = []
train_accuracies = []
val_accuracies = []

for epoch in range(best_params['num_epochs']):
    model.train()
    for batch_start in range(0, num_samples, best_params['batch_size']):
        batch_end = min(batch_start + best_params['batch_size'], num_samples)
        X_batch = X_train[batch_start:batch_end]
        y_batch = y_train[batch_start:batch_end]

        optimizer.zero_grad()
        y_pred = model(X_train)

        loss = best_params['criterion'](y_pred, y_train)
        loss.backward()

        optimizer.step()

    model.eval()
    with torch.no_grad():
        train_predictions = (model(X_train) > 0.5).float()
        val_predictions = (model(X_test) > 0.5).float()

    train_loss = best_params['criterion'](train_predictions, y_train.view(-1, 1))
    val_loss = best_params['criterion'](val_predictions, y_test.view(-1, 1))

    train_losses.append(train_loss.item())
    val_losses.append(val_loss.item())

    train_accuracy = accuracy_score(y_train.cpu().numpy(),
train_predictions.cpu().numpy())
    val_accuracy = accuracy_score(y_test.cpu().numpy(), val_predictions.cpu().numpy())

```

```

train_accuracies.append(train_accuracy)
val_accuracies.append(val_accuracy)

#print(val_accuracy)

print(f'Epoch : {epoch+1}, Training Loss : {train_loss}, Validation Loss : {val_loss}')

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(range(1, best_params['num_epochs'] + 1), train_losses, label='Training Hinge Loss')
plt.plot(range(1, best_params['num_epochs'] + 1), val_losses, label='Validation Hinge Loss')
plt.title('Training and Validation Hinge Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Hinge Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(1, best_params['num_epochs'] + 1), train_accuracies, label='Training Accuracy')
plt.plot(range(1, best_params['num_epochs'] + 1), val_accuracies, label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.tight_layout()
plt.show()

model.eval()

with torch.no_grad():
    y_prob_tensor = model(X_test)
    y_pred_tensor = (y_prob_tensor > 0.5).float()

y_pred = y_pred_tensor.cpu().detach().numpy()
y_prob = y_prob_tensor.cpu().detach().numpy()

test_accuracy = accuracy_score(y_test.cpu().detach().numpy(), y_pred)
print("Testing Accuracy:", test_accuracy)

```

```

print("Classification Report:")
print(classification_report(y_test.cpu().detach().numpy(), y_pred))

fpr, tpr, _ = roc_curve(y_test.cpu().detach().numpy(), y_prob)
auc_score = roc_auc_score(y_test.cpu().detach().numpy(), y_prob)

plt.figure(figsize=(8, 8))
plt.plot(fpr, tpr, label=f'AUC = {auc_score:.2f}')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.title('ROC Curve')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()

plt.show()

```

2. Wisconsin Breast Cancer Dataset

2.1. Logistic Regression with Regularization

```

import numpy as np
from ucimlrepo import fetch_ucirepo
import matplotlib.pyplot as plt
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from sklearn.linear_model import SGDClassifier
from sklearn.kernel_approximation import RBFSampler
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score, classification_report, roc_curve
from sklearn.metrics import roc_auc_score, log_loss, confusion_matrix
import seaborn as sns

# Fetch dataset
breast_cancer_wisconsin_diagnostic = fetch_ucirepo(id=17)

# Data (as pandas dataframes)
X = breast_cancer_wisconsin_diagnostic.data.features
y = breast_cancer_wisconsin_diagnostic.data.targets

# Standardizing the data
scaler = StandardScaler()
X = scaler.fit_transform(X)
class_mapping = {'M': 1, 'B': 0}

```

```

y = [class_mapping[target] for target in y.T.values[0]]

# Splitting the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define the model
model = SGDClassifier(random_state=42)

# Define the hyperparameter grid
param_grid = {
    'alpha': [0.0001, 0.001, 0.01, 0.1, 1],
    'penalty': ['l1', 'l2'],
    'max_iter': [1000],
    'loss': ['log_loss'],
    'learning_rate': ['constant'],
    'eta0': [0.0001, 0.001, 0.01, 0.1]
}

# Create GridSearchCV object with cross-validation
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)

# Get the best parameters and the corresponding model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

all_models = grid_search.cv_results_['params']

# Use cross_val_score to get cross-validation scores for each model
cross_val_scores = []
for model_params in all_models:
    model = SGDClassifier(**model_params, random_state=42)
    scores = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy')
    cross_val_scores.append(scores)

plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)
plt.xticks(ticks=range(len(all_models)))

# Simulate partial_fit with batch-wise training
batch_size = X_train.shape[0]
num_samples = X_train.shape[0]
num_batches = int(np.ceil(num_samples / batch_size))

```

```

print("Best model : ", best_params)

# Training and validation losses
epochs = [5, 8, 10]
for each in epochs:
    train_losses = []
    val_losses = []
    train_accuracies = []
    val_accuracies = []

    # best_sgd_model = SGDClassifier(**best_params)
    best_sgd_model = SGDClassifier(alpha=best_params['alpha'], eta0=best_params['eta0'],
                                   learning_rate=best_params['learning_rate'], loss=best_params['loss'],

max_iter=best_params['max_iter'], penalty=best_params['penalty'], random_state=42)

    # Learning rate schedule
    initial_learning_rate = 0.01
    learning_rate = initial_learning_rate

    # Early stopping parameters
    patience = 5
    best_val_loss = float('inf')
    no_improvement_count = 0

    for epoch in range(each):
        for batch_start in range(0, num_samples, batch_size):
            batch_end = min(batch_start + batch_size, num_samples)
            X_batch = X_train[batch_start:batch_end]
            y_batch = y_train[batch_start:batch_end]

            # Train the model on the batch
            best_sgd_model.partial_fit(X_batch, y_batch, classes=np.unique(y_train))

        # Evaluate on training and validation sets
        train_predictions = best_sgd_model.predict(X_train)
        val_predictions = best_sgd_model.predict(X_test)
        train_loss = log_loss(y_train, train_predictions)
        val_loss = log_loss(y_test, val_predictions)
        train_losses.append(train_loss)
        val_losses.append(val_loss)
        train_accuracy = accuracy_score(y_train, train_predictions)
        val_accuracy = accuracy_score(y_test, val_predictions)
        train_accuracies.append(train_accuracy)

```

```

val_accuracies.append(val_accuracy)

# Learning rate schedule (decay every 10 epochs)
if epoch % 10 == 0:
    learning_rate = learning_rate * 0.9

# Plotting
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(range(1, len(train_losses) + 1), train_losses, label='Training Log Loss')
plt.plot(range(1, len(val_losses) + 1), val_losses, label='Validation Log Loss')
plt.title('Training and Validation Log Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Log Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(1, len(train_accuracies) + 1), train_accuracies, label='Training Accuracy')
plt.plot(range(1, len(val_accuracies) + 1), val_accuracies, label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.tight_layout()
plt.show()

# Evaluate the best model on the test set
y_pred = best_sgd_model.predict(X_test)
y_prob = best_sgd_model.decision_function(X_test)

# Testing accuracy
test_accuracy = accuracy_score(y_test, y_pred)
print("Testing Accuracy:", test_accuracy)

# Classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))

# Confusion matrix
confusion_mat = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(confusion_mat)

```

```

# ROC curve and AUC score
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc_score = roc_auc_score(y_test, y_prob)
plt.figure(figsize=(8, 8))
plt.plot(fpr, tpr, label=f'AUC = {auc_score:.2f}')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.title('ROC Curve')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.show()

```

2.2. Logistic Regression without Regularization

```

import numpy as np
from ucimlrepo import fetch_ucirepo
import matplotlib.pyplot as plt
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from sklearn.linear_model import SGDClassifier
from sklearn.kernel_approximation import RBFSampler
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score, classification_report, roc_curve
from sklearn.metrics import roc_auc_score, log_loss, confusion_matrix
import seaborn as sns

# Fetch dataset
breast_cancer_wisconsin_diagnostic = fetch_ucirepo(id=17)

# Data (as pandas dataframes)
X = breast_cancer_wisconsin_diagnostic.data.features
y = breast_cancer_wisconsin_diagnostic.data.targets

# Standardizing the data
scaler = StandardScaler()
X = scaler.fit_transform(X)
class_mapping = {'M': 1, 'B': 0}
y = [class_mapping[target] for target in y.T.values[0]]

# Splitting the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

```

# Define the model
model = SGDClassifier(random_state=42)

# Define the hyperparameter grid
param_grid = {
    'alpha': [0.0001, 0.001, 0.01, 0.1, 1],
    'penalty': [None],
    'max_iter': [1000],
    'loss' : ['log_loss'],
    'learning_rate' : ['constant'],
    'eta0' : [0.0001, 0.001, 0.01, 0.1]
}

# Create GridSearchCV object with cross-validation
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)

# Get the best parameters and the corresponding model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

all_models = grid_search.cv_results_['params']

# Use cross_val_score to get cross-validation scores for each model
cross_val_scores = []
for model_params in all_models:
    model = SGDClassifier(**model_params, random_state=42)
    scores = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy')
    cross_val_scores.append(scores)

plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)
plt.xticks(ticks=range(len(all_models)))

# Simulate partial_fit with batch-wise training
batch_size = X_train.shape[0]
num_samples = X_train.shape[0]
num_batches = int(np.ceil(num_samples / batch_size))
print("Best model : ", best_params)

# Training and validation losses
epochs = [5, 8, 10]
for each in epochs:

```



```

train_losses = []
val_losses = []
train_accuracies = []
val_accuracies = []

# best_sgd_model = SGDClassifier(**best_params)
best_sgd_model = SGDClassifier(alpha=best_params['alpha'], eta0=best_params['eta0'],
                               learning_rate=best_params['learning_rate'], loss=best_params['loss'],
max_iter=best_params['max_iter'], penalty=best_params['penalty'], random_state=42)

# Learning rate schedule
initial_learning_rate = 0.01
learning_rate = initial_learning_rate

# Early stopping parameters
patience = 5
best_val_loss = float('inf')
no_improvement_count = 0

for epoch in range(each):
    for batch_start in range(0, num_samples, batch_size):
        batch_end = min(batch_start + batch_size, num_samples)
        X_batch = X_train[batch_start:batch_end]
        y_batch = y_train[batch_start:batch_end]

        # Train the model on the batch
        best_sgd_model.partial_fit(X_batch, y_batch, classes=np.unique(y_train))

    # Evaluate on training and validation sets
    train_predictions = best_sgd_model.predict(X_train)
    val_predictions = best_sgd_model.predict(X_test)
    train_loss = log_loss(y_train, train_predictions)
    val_loss = log_loss(y_test, val_predictions)
    train_losses.append(train_loss)
    val_losses.append(val_loss)
    train_accuracy = accuracy_score(y_train, train_predictions)
    val_accuracy = accuracy_score(y_test, val_predictions)
    train_accuracies.append(train_accuracy)
    val_accuracies.append(val_accuracy)

# Learning rate schedule (decay every 10 epochs)
if epoch % 10 == 0:
    learning_rate = learning_rate * 0.9

```

```

# Plotting
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(range(1, len(train_losses) + 1), train_losses, label='Training Log Loss')
plt.plot(range(1, len(val_losses) + 1), val_losses, label='Validation Log Loss')
plt.title('Training and Validation Log Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Log Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(1, len(train_accuracies) + 1), train_accuracies, label='Training Accuracy')
plt.plot(range(1, len(val_accuracies) + 1), val_accuracies, label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.tight_layout()
plt.show()

# Evaluate the best model on the test set
y_pred = best_sgd_model.predict(X_test)
y_prob = best_sgd_model.decision_function(X_test)

# Testing accuracy
test_accuracy = accuracy_score(y_test, y_pred)
print("Testing Accuracy:", test_accuracy)

# Classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))

# Confusion matrix
confusion_mat = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(confusion_mat)

# ROC curve and AUC score
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc_score = roc_auc_score(y_test, y_prob)

```

```

plt.figure(figsize=(8, 8))
plt.plot(fpr, tpr, label=f'AUC = {auc_score:.2f}')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.title('ROC Curve')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.show()

```

2.3. SVM with Kernels

```
!pip install ucimlrepo
```

```

from ucimlrepo import fetch_ucirepo
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV, cross_val_score
from sklearn.linear_model import SGDClassifier
from sklearn.kernel_approximation import RBFSampler
from sklearn.metrics import accuracy_score, classification_report, roc_curve, roc_auc_score,
hinge_loss
import matplotlib.pyplot as plt
import numpy as np

# Fetch dataset
breast_cancer_wisconsin_diagnostic = fetch_ucirepo(id=17)

# Data (as pandas dataframes)
X = breast_cancer_wisconsin_diagnostic.data.features
y = breast_cancer_wisconsin_diagnostic.data.targets

# Convert labels to binary (M: 1, B: 0)
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize features
scalar = StandardScaler()
sc_X_train = scalar.fit_transform(X_train)
sc_X_test = scalar.transform(X_test)

```

```

# Apply PCA
pca = PCA(n_components=23) # You can adjust the explained variance threshold
X_train_pca = pca.fit_transform(sc_X_train)
X_test_pca = pca.transform(sc_X_test)

# Define the RBF Sampler
rbf_sampler = RBFSampler(gamma=0.001, random_state=42) # Initial gamma, you can change it later

# Apply kernel approximation to the training and testing data
X_train_transformed = rbf_sampler.fit_transform(X_train_pca)
X_test_transformed = rbf_sampler.transform(X_test_pca)

# Define the model
model = SGDClassifier(random_state=42)

# Define the hyperparameter grid
param_grid = {
    'alpha': [0.0001, 0.01],
    'penalty': ['l1', 'l2'],
    'max_iter': [1000],
    'loss': ['hinge'],
    'learning_rate': ['constant'],
    'eta0': [0.0001, 0.001, 0.01]
}

# Create GridSearchCV object with cross-validation
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train_transformed, y_train)

# Get the best parameters and the corresponding model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split

all_models = grid_search.cv_results_['params']

# Use cross_val_score to get cross-validation scores for each model
cross_val_scores = []
for model_params in all_models:
    model = SGDClassifier(**model_params)

```

```
scores = cross_val_score(model, X_train_transformed, y_train, cv=5, scoring='accuracy')
cross_val_scores.append(scores)
```

```
plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)
plt.xticks(ticks=range(len(all_models)), labels=['Model' + str(i+1) for i in range(len(all_models))],
rotation=45, ha='right')
plt.title('Cross-Validation Scores for Each Model')
plt.xlabel('Model Parameters')
plt.ylabel('Accuracy')
plt.show()
```

```
# Simulate partial_fit with batch-wise training
batch_size = 32
num_samples = X_train_transformed.shape[0]
num_batches = int(np.ceil(num_samples / batch_size))
```

```
print("Best model : ", best_params)
```

```
# Training and validation losses
```

```
epochs = 60
train_losses = []
val_losses = []
train_accuracies = []
val_accuracies = []
```

```
for epoch in range(epochs):
```

```
    for batch_start in range(0, num_samples, batch_size):
        batch_end = min(batch_start + batch_size, num_samples)
        X_batch = X_train_transformed[batch_start:batch_end]
        y_batch = y_train[batch_start:batch_end]
```

```
        # Train the model on the batch
```

```
        best_model.partial_fit(X_batch, y_batch, classes=np.unique(y_train))
```

```
# Evaluate on training and validation sets
```

```
train_predictions = best_model.predict(X_train_transformed)
```

```
val_predictions = best_model.predict(X_test_transformed)
```

```
train_loss = hinge_loss(y_train, train_predictions)
```

```
val_loss = hinge_loss(y_test, val_predictions)
```

```
train_losses.append(train_loss)
```

```
val_losses.append(val_loss)
```

```

train_accuracy = accuracy_score(y_train, train_predictions)
val_accuracy = accuracy_score(y_test, val_predictions)

train_accuracies.append(train_accuracy)
val_accuracies.append(val_accuracy)

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(range(1, epochs + 1), train_losses, label='Training Hinge Loss')
plt.plot(range(1, epochs + 1), val_losses, label='Validation Hinge Loss')
plt.title('Training and Validation Hinge Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Hinge Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(1, epochs + 1), train_accuracies, label='Training Accuracy')
plt.plot(range(1, epochs + 1), val_accuracies, label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.tight_layout()
plt.show()

# Evaluate the best model on the test set
y_pred = best_model.predict(X_test_transformed)
y_prob = best_model.decision_function(X_test_transformed)

# Testing accuracy
test_accuracy = accuracy_score(y_test, y_pred)
print("Testing Accuracy:", test_accuracy)

# Classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))

# ROC curve and AUC score
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc_score = roc_auc_score(y_test, y_prob)

plt.figure(figsize=(8, 8))

```

```
plt.plot(fpr, tpr, label=f'AUC = {auc_score:.2f}')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.title('ROC Curve')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.show()
```

2.4. SVM without Kernels

```
!pip install ucimlrepo
```

```
from ucimlrepo import fetch_ucirepo
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV, cross_val_score
from sklearn.linear_model import SGDClassifier
from sklearn.kernel_approximation import RBFSampler
from sklearn.metrics import accuracy_score, classification_report, roc_curve, roc_auc_score,
hinge_loss
import matplotlib.pyplot as plt
import numpy as np

# Fetch dataset
breast_cancer_wisconsin_diagnostic = fetch_ucirepo(id=17)

# Data (as pandas dataframes)
X = breast_cancer_wisconsin_diagnostic.data.features
y = breast_cancer_wisconsin_diagnostic.data.targets

# Convert labels to binary (M: 1, B: 0)
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize features
scalar = StandardScaler()
sc_X_train = scalar.fit_transform(X_train)
sc_X_test = scalar.transform(X_test)
```

```

# Apply PCA
pca = PCA(n_components=23) # You can adjust the explained variance threshold
X_train_pca = pca.fit_transform(sc_X_train)
X_test_pca = pca.transform(sc_X_test)

# Define the RBF Sampler
rbf_sampler = RBFSampler(gamma=0.001, random_state=42) # Initial gamma, you can change it later

# Define the model
model = SGDClassifier(random_state=42)

# Define the hyperparameter grid
param_grid = {
    'alpha': [0.0001, 0.01],
    'penalty': ['l1', 'l2'],
    'max_iter': [1000],
    'loss': ['hinge'],
    'learning_rate': ['constant'],
    'eta0': [0.0001, 0.001, 0.01]
}

X_train_transformed = X_train_pca
X_test_transformed = X_test_pca

# Create GridSearchCV object with cross-validation
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train_transformed, y_train)

# Get the best parameters and the corresponding model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split

all_models = grid_search.cv_results_['params']

# Use cross_val_score to get cross-validation scores for each model
cross_val_scores = []
for model_params in all_models:
    model = SGDClassifier(**model_params)
    scores = cross_val_score(model, X_train_transformed, y_train, cv=5, scoring='accuracy')
    cross_val_scores.append(scores)

```



```

plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)
plt.xticks(ticks=range(len(all_models)), labels=['Model' + str(i+1) for i in range(len(all_models))],
rotation=45, ha='right')
plt.title('Cross-Validation Scores for Each Model')
plt.xlabel('Model Parameters')
plt.ylabel('Accuracy')
plt.show()

```

```

# Simulate partial_fit with batch-wise training
batch_size = 32
num_samples = X_train_transformed.shape[0]
num_batches = int(np.ceil(num_samples / batch_size))

```

```

print("Best model : ", best_params)

```

```

# Training and validation losses

```

```

epochs = 60
train_losses = []
val_losses = []
train_accuracies = []
val_accuracies = []

```

```

for epoch in range(epochs):

```

```

    for batch_start in range(0, num_samples, batch_size):
        batch_end = min(batch_start + batch_size, num_samples)
        X_batch = X_train_transformed[batch_start:batch_end]
        y_batch = y_train[batch_start:batch_end]

```

```

        # Train the model on the batch

```

```

        best_model.partial_fit(X_batch, y_batch, classes=np.unique(y_train))

```

```

# Evaluate on training and validation sets

```

```

train_predictions = best_model.predict(X_train_transformed)
val_predictions = best_model.predict(X_test_transformed)

```

```

train_loss = hinge_loss(y_train, train_predictions)
val_loss = hinge_loss(y_test, val_predictions)

```

```

train_losses.append(train_loss)
val_losses.append(val_loss)

```

```

train_accuracy = accuracy_score(y_train, train_predictions)

```

```

val_accuracy = accuracy_score(y_test, val_predictions)

train_accuracies.append(train_accuracy)
val_accuracies.append(val_accuracy)

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(range(1, epochs + 1), train_losses, label='Training Hinge Loss')
plt.plot(range(1, epochs + 1), val_losses, label='Validation Hinge Loss')
plt.title('Training and Validation Hinge Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Hinge Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(1, epochs + 1), train_accuracies, label='Training Accuracy')
plt.plot(range(1, epochs + 1), val_accuracies, label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.tight_layout()
plt.show()

# Evaluate the best model on the test set
y_pred = best_model.predict(X_test_transformed)
y_prob = best_model.decision_function(X_test_transformed)

# Testing accuracy
test_accuracy = accuracy_score(y_test, y_pred)
print("Testing Accuracy:", test_accuracy)

# Classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))

# ROC curve and AUC score
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc_score = roc_auc_score(y_test, y_prob)

plt.figure(figsize=(8, 8))
plt.plot(fpr, tpr, label=f'AUC = {auc_score:.2f}')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')

```

```
plt.title('ROC Curve')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.show()
```

2.5. PCA + k-NN

```
import numpy as np
from ucimlrepo import fetch_ucirepo
import matplotlib.pyplot as plt
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from sklearn.linear_model import SGDClassifier
from sklearn.kernel_approximation import RBFSampler
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score, classification_report, roc_curve
from sklearn.metrics import roc_auc_score, log_loss, confusion_matrix
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
import seaborn as sns
```

```
# Fetch dataset
breast_cancer_wisconsin_diagnostic = fetch_ucirepo(id=17)
```

```
# Data (as pandas dataframes)
X = breast_cancer_wisconsin_diagnostic.data.features
y = breast_cancer_wisconsin_diagnostic.data.targets
```

```
# Standardizing the data
scaler = StandardScaler()
X = scaler.fit_transform(X)
class_mapping = {'M': 1, 'B': 0}
y = [class_mapping[target] for target in y.T.values[0]]
```

```
# Splitting the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
max_components = min(X_test.shape[0], X_test.shape[1])
print("Maximum number of components:", max_components)
```

```
num_components = [10, 15, 20]
for every in num_components:
    # Apply PCA for dimensionality reduction (optional)
```

```

# You can skip this step or adjust the number of components based on your needs
# num_components = 15 # You can choose the number of components you want
pca = PCA(n_components=every)
# Fit and transform the training data
X_train_transformed = pca.fit_transform(X_train)
# Transform the test data
X_test_transformed = pca.transform(X_test)
# Visualize the first two principal components
explained_var_ratio = pca.explained_variance_ratio_
# Scree plot
plt.figure(figsize=(10, 6))
plt.bar(range(1, every + 1), explained_var_ratio, alpha=0.7, align='center')
plt.title('Explained Variance Ratio for Each Principal Component')
plt.xlabel('Principal Component')
plt.ylabel('Explained Variance Ratio')
plt.show()

# Define the model
model = KNeighborsClassifier()

# Define the hyperparameter grid
param_grid = {
    'n_neighbors': [2,3,4,5,6,7,8],
    'weights': ['uniform','distance'],
    'p': [1,2]
}

# Create GridSearchCV object with cross-validation
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train_transformed, y_train)

# Get the best parameters and the corresponding model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

all_models = grid_search.cv_results_['params']

# Use cross_val_score to get cross-validation scores for each model
cross_val_scores = []
for model_params in all_models:
    model = KNeighborsClassifier(**model_params)
    scores = cross_val_score(model, X_train_transformed, y_train, cv=5, scoring='accuracy')
    cross_val_scores.append(scores)

```

```

plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)
plt.xticks(ticks=range(len(all_models)))

# Simulate partial_fit with batch-wise training
batch_size = X_train.shape[0]
num_samples = X_train.shape[0]
num_batches = int(np.ceil(num_samples / batch_size))
print("Best model : ", best_params)

# best_sgd_model = SGDClassifier(**best_params)
best_knn_model = KNeighborsClassifier(n_neighbors=best_params['n_neighbors'])

# Train the model
best_knn_model.fit(X_train_transformed, y_train)

# Evaluate the best model on the test set
y_pred = best_knn_model.predict(X_test_transformed)
y_prob = best_knn_model.predict_proba(X_test_transformed)

# Testing accuracy
test_accuracy = accuracy_score(y_test, y_pred)
print("Testing Accuracy:", test_accuracy)

# Classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))

# Confusion matrix
confusion_mat = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(confusion_mat)

```

2.6. FNN

```

!pip install ucimlrepo

from ucimlrepo import fetch_ucirepo

breast_cancer_wisconsin_diagnostic = fetch_ucirepo(id=17)

X = breast_cancer_wisconsin_diagnostic.data.features

```

```

y = breast_cancer_wisconsin_diagnostic.data.targets

from sklearn.preprocessing import StandardScaler
import numpy as np
from sklearn.model_selection import train_test_split

scaler = StandardScaler()

X = scaler.fit_transform(X)

class_mapping = {'M': 1, 'B': 0}

y = [class_mapping[target] for target in y.T.values[0]]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score

input_size = 30
output_size = 1
num_epochs = 50
batch_size = 32

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

X_train = torch.tensor(X_train,
dtype=torch.float32).clone().detach().to(device).type(torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).clone().detach().view(-1,
1).to(device).type(torch.float32)
X_test = torch.tensor(X_test,
dtype=torch.float32).clone().detach().to(device).type(torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32).clone().detach().view(-1,
1).to(device).type(torch.float32)

class FNN(nn.Module):
    def __init__(self, input_size, hidden_sizes, output_size, dropout_prob=0.4):
        super(FNN, self).__init__()
        layer_sizes = [input_size] + hidden_sizes + [output_size]

```

```

        layers.append(nn.Linear(layer_sizes[i], layer_sizes[i + 1]))
        if i < len(layer_sizes) - 1:
            layers.append(nn.ReLU()) # Adding ReLU activation between hidden layers
        if i < len(layer_sizes) - 2:
            layers.append(nn.Dropout(dropout_prob)) # Dropout layer between hidden
layers
        layers.append(nn.Sigmoid()) # Sigmoid activation for the output layer
        self.model = nn.Sequential(*layers)

    def forward(self, x):
        return self.model(x)

```

```

from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split

```

```

param_grid = {
    'lr': [0.01, 0.001],
    'hidden_sizes': [[32, 16], [64, 16], [64, 8]],
    'input_size': [X_train.shape[1]],
    'output_size': [1],
    'criterion': [nn.BCELoss()],
    'optimizer_class': [optim.Adam],
    'wd': [0.001, 0.01],
    'num_epochs': [50],
    'batch_size': [32, X_train.shape[0]],
    'model_class' : [FNN]
}

```

```

class PyTorchEstimator(BaseEstimator, ClassifierMixin):
    def __init__(self, model_class, criterion, optimizer_class, input_size=30, hidden_sizes
= [32, 16], output_size = 1, lr=0.001, wd=0.0, num_epochs=20, batch_size=32, model =
None):
        self.model_class = model_class
        self.criterion = criterion
        self.optimizer_class = optimizer_class
        self.input_size = input_size
        self.hidden_sizes = hidden_sizes
        self.output_size = output_size
        self.lr = lr
        self.wd = wd

```

```

self.num_epochs = num_epochs
self.batch_size = batch_size
self.model = None

def fit(self, X, y):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    X_tensor = torch.tensor(X, dtype=torch.float32).to(device)
    y_tensor = torch.tensor(y, dtype=torch.float32).view(-1, 1).to(device)

    num_samples = X.shape[0]

    self.model = self.model_class(self.input_size, self.hidden_sizes,
self.output_size).to(device)
    optimizer = self.optimizer_class(self.model.parameters(), lr=self.lr,
weight_decay=self.wd)

    for epoch in range(self.num_epochs):
        self.model.train()
        for batch_start in range(0, num_samples, self.batch_size):
            batch_end = min(batch_start + self.batch_size, num_samples)
            X_batch = X_tensor[batch_start:batch_end]
            y_batch = y_tensor[batch_start:batch_end]

            optimizer.zero_grad()
            y_pred = self.model(X_batch)

            loss = self.criterion(y_pred, y_batch)
            loss.backward()

            optimizer.step()

        if epoch % 10 == 0:
            print(f'Epoch {epoch + 1} : Parameters - Batch size: {self.batch_size}, lr:
{self.lr}, wd: {self.wd}')

    return self

def predict(self, X):
    self.model.eval()
    with torch.no_grad():
        X_tensor = torch.tensor(X, dtype=torch.float32).to(device)
        y_pred = self.model(X_tensor)
        y_pred_binary = (y_pred > 0.5).float()
    return y_pred_binary.cpu().numpy()

```



```

def get_params(self, deep=True):
    return {
        'model_class': self.model_class,
        'criterion': self.criterion,
        'optimizer_class': self.optimizer_class,
        'input_size': self.input_size,
        'hidden_sizes': self.hidden_sizes,
        'output_size': self.output_size,
        'lr': self.lr,
        'num_epochs': self.num_epochs,
        'batch_size': self.batch_size,
        'model': self.model,
        'wd': self.wd
    }

```

```

pytorch_estimator = PyTorchEstimator(FNN, nn.BCELoss(), optim.Adam)
#print(X_train.device, y_train.device)
grid_search = GridSearchCV(pytorch_estimator, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train.cpu(), y_train.cpu())

```

```

best_params = grid_search.best_params_

```

```

import seaborn as sns
import matplotlib.pyplot as plt

```

```

all_models = grid_search.cv_results_['params']

```

```

cross_val_scores = []
for model_params in all_models:
    model = PyTorchEstimator(**model_params)
    scores = cross_val_score(model, X_train.cpu(), y_train.cpu(), cv=5,
scoring='accuracy')
    cross_val_scores.append(scores)

```

```

plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)
plt.xticks(ticks=range(len(all_models)), labels=['Model' + str(i+1) for i in
range(len(all_models))], rotation=45, ha='right')
plt.title('Cross-Validation Scores for Each Model')
plt.xlabel('Model Parameters')
plt.ylabel('Accuracy')

```

```

plt.show()
print("Best model : ", best_params)

import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, classification_report, roc_curve,
roc_auc_score
import matplotlib.pyplot as plt

num_samples = X_train.shape[0]
num_batches = int(np.ceil(num_samples / num_samples))

model = best_params['model_class'](best_params['input_size'],
best_params['hidden_sizes'], best_params['output_size']).to(device)
optimizer = best_params['optimizer_class'](model.parameters(), lr=best_params['lr'],
weight_decay = best_params['wd'])

train_losses = []
val_losses = []
train_accuracies = []
val_accuracies = []

for epoch in range(best_params['num_epochs']):
    model.train()
    for batch_start in range(0, num_samples, best_params['batch_size']):
        batch_end = min(batch_start + best_params['batch_size'], num_samples)
        X_batch = X_train[batch_start:batch_end]
        y_batch = y_train[batch_start:batch_end]

        optimizer.zero_grad()
        y_pred = model(X_train)

        loss = best_params['criterion'](y_pred, y_train)
        loss.backward()

        optimizer.step()

    model.eval()
    with torch.no_grad():
        train_predictions = (model(X_train) > 0.5).float()
        val_predictions = (model(X_test) > 0.5).float()

train_loss = best_params['criterion'](train_predictions, y_train.view(-1, 1))

```

```

val_loss = best_params['criterion'](val_predictions, y_test.view(-1, 1))

train_losses.append(train_loss.item())
val_losses.append(val_loss.item())

train_accuracy = accuracy_score(y_train.cpu().numpy(),
train_predictions.cpu().numpy())
val_accuracy = accuracy_score(y_test.cpu().numpy(), val_predictions.cpu().numpy())

train_accuracies.append(train_accuracy)
val_accuracies.append(val_accuracy)

#print(val_accuracy)

print(f'Epoch : {epoch+1}, Training Loss : {train_loss}, Validation Loss : {val_loss}')

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(range(1, best_params['num_epochs'] + 1), train_losses, label='Training Hinge Loss')
plt.plot(range(1, best_params['num_epochs'] + 1), val_losses, label='Validation Hinge Loss')
plt.title('Training and Validation Hinge Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel(' Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(1, best_params['num_epochs'] + 1), train_accuracies, label='Training Accuracy')
plt.plot(range(1, best_params['num_epochs'] + 1), val_accuracies, label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.tight_layout()
plt.show()

model.eval()

with torch.no_grad():
    y_prob_tensor = model(X_test)

```

```

y_pred_tensor = (y_prob_tensor > 0.5).float()

y_pred = y_pred_tensor.cpu().detach().numpy()
y_prob = y_prob_tensor.cpu().detach().numpy()

test_accuracy = accuracy_score(y_test.cpu().detach().numpy(), y_pred)
print("Testing Accuracy:", test_accuracy)

print("Classification Report:")
print(classification_report(y_test.cpu().detach().numpy(), y_pred))

fpr, tpr, _ = roc_curve(y_test.cpu().detach().numpy(), y_prob)
auc_score = roc_auc_score(y_test.cpu().detach().numpy(), y_prob)

plt.figure(figsize=(8, 8))
plt.plot(fpr, tpr, label=f'AUC = {auc_score:.2f}')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.title('ROC Curve')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()

plt.show()

```

3. Fashion MNIST Dataset

3.1. Logistic Regression with Regularization

```

!pip install tensorflow
from sklearn.datasets import fetch_openml
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDClassifier
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import fashion_mnist
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from sklearn.kernel_approximation import RBFSampler
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score, classification_report, roc_curve
from sklearn.metrics import roc_auc_score, log_loss, confusion_matrix
import seaborn as sns
import pandas as pd

```

```

from sklearn import datasets
from sklearn.svm import SVC
from sklearn.decomposition import PCA

# Load and preprocess the Fashion MNIST dataset
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()
X_train = X_train.reshape((60000, 28, 28)).astype('float32')
X_test = X_test.reshape((10000, 28, 28)).astype('float32')
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

# Load Fashion MNIST dataset
# fashion_mnist = datasets.fetch_openml('Fashion-MNIST', version=1, cache=True)
X_train = X_train.reshape(60000, 28*28)
X_test = X_test.reshape(10000, 28*28)
# Standardize the features
scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)

# Apply PCA for dimensionality reduction (optional)
# You can skip this step or adjust the number of components based on your needs
num_components = 50 # You can choose the number of components you want
pca = PCA(n_components=num_components)
# Fit and transform the training data
X_train = pca.fit_transform(X_train_std)
# Transform the test data
X_test = pca.transform(X_test_std)
# Visualize the first two principal components
explained_var_ratio = pca.explained_variance_ratio_
# Scree plot
plt.figure(figsize=(10, 6))
plt.bar(range(1, num_components + 1), explained_var_ratio, alpha=0.7, align='center')
plt.title('Explained Variance Ratio for Each Principal Component')
plt.xlabel('Principal Component')
plt.ylabel('Explained Variance Ratio')
plt.show()

# Define the model
model = SGDClassifier(random_state=42)

# Define the hyperparameter grid
param_grid = {
    'alpha': [0.00001, 0.0001, 0.001, 0.01],

```

```
'penalty': ['l1', 'l2'],
'max_iter': [1000],
'loss' : ['log_loss'],
'learning_rate' : ['constant'],
'eta0' : [0.0001, 0.001, 0.01, 0.1]
}
```

```
# Create GridSearchCV object with cross-validation
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)
```

```
# Get the best parameters and the corresponding model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_
```

```
all_models = grid_search.cv_results_['params']
```

```
# Use cross_val_score to get cross-validation scores for each model
cross_val_scores = []
for model_params in all_models:
    model = SGDClassifier(**model_params, random_state=42)
    scores = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy')
    cross_val_scores.append(scores)
```

```
plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)
plt.xticks(ticks=range(len(all_models)))
```

```
# Simulate partial_fit with batch-wise training
batch_size = X_train.shape[0]
num_samples = X_train.shape[0]
num_batches = int(np.ceil(num_samples / batch_size))
print("Best model : ", best_params)
```

```
# Training and validation losses
epochs = [60, 80]
for each in epochs:
    train_losses = []
    val_losses = []
    train_accuracies = []
    val_accuracies = []
```

```
# best_sgd_model = SGDClassifier(**best_params)
```

```

best_sgd_model = SGDClassifier(alpha=best_params['alpha'], eta0=best_params['eta0'],
                               learning_rate=best_params['learning_rate'], loss=best_params['loss'],
max_iter=best_params['max_iter'], penalty=best_params['penalty'], random_state=42)

# Learning rate schedule
initial_learning_rate = 0.01
learning_rate = initial_learning_rate

# Early stopping parameters
patience = 5
best_val_loss = float('inf')
no_improvement_count = 0

for epoch in range(each):
    for batch_start in range(0, num_samples, batch_size):
        batch_end = min(batch_start + batch_size, num_samples)
        X_batch = X_train[batch_start:batch_end]
        y_batch = y_train[batch_start:batch_end]

        # Train the model on the batch
        best_sgd_model.partial_fit(X_batch, y_batch, classes=np.unique(y_train))
    # print(epoch)

# Evaluate on training and validation sets
train_predictions = best_sgd_model.predict(X_train)
val_predictions = best_sgd_model.predict(X_test)
train_probability = best_sgd_model.predict_proba(X_train)
val_probability = best_sgd_model.predict_proba(X_test)
# print(y_train)
# print(train_predictions)
train_loss = log_loss(y_train, train_probability)
val_loss = log_loss(y_test, val_probability)
train_losses.append(train_loss)
val_losses.append(val_loss)
train_accuracy = accuracy_score(y_train, train_predictions)
val_accuracy = accuracy_score(y_test, val_predictions)
train_accuracies.append(train_accuracy)
val_accuracies.append(val_accuracy)

# # Learning rate schedule (decay every 10 epochs)
# if epoch % 10 == 0:
#     learning_rate = learning_rate * 0.9

```

```

# Plotting
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(range(1, len(train_losses) + 1), train_losses, label='Training Log Loss')
plt.plot(range(1, len(val_losses) + 1), val_losses, label='Validation Log Loss')
plt.title('Training and Validation Log Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Log Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(1, len(train_accuracies) + 1), train_accuracies, label='Training Accuracy')
plt.plot(range(1, len(val_accuracies) + 1), val_accuracies, label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.tight_layout()
plt.show()

# Evaluate the best model on the test set
y_pred = best_sgd_model.predict(X_test)
y_prob = best_sgd_model.decision_function(X_test)

# Testing accuracy
test_accuracy = accuracy_score(y_test, y_pred)
print("Testing Accuracy:", test_accuracy)

# Classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))

# Confusion matrix
confusion_mat = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(confusion_mat)

```

3.2. Logistic Regression without Regularization

```

!pip install tensorflow
from sklearn.datasets import fetch_openml

```



```

from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDClassifier
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import fashion_mnist
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from sklearn.kernel_approximation import RBFSampler
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score, classification_report, roc_curve
from sklearn.metrics import roc_auc_score, log_loss, confusion_matrix
import seaborn as sns
import pandas as pd
from sklearn import datasets
from sklearn.svm import SVC
from sklearn.decomposition import PCA

# Load and preprocess the Fashion MNIST dataset
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()
X_train = X_train.reshape((60000, 28, 28)).astype('float32')
X_test = X_test.reshape((10000, 28, 28)).astype('float32')
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

# Load Fashion MNIST dataset
# fashion_mnist = datasets.fetch_openml('Fashion-MNIST', version=1, cache=True)
X_train = X_train.reshape(60000, 28*28)
X_test = X_test.reshape(10000, 28*28)
# Standardize the features
scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)

# Apply PCA for dimensionality reduction (optional)
# You can skip this step or adjust the number of components based on your needs
num_components = 50 # You can choose the number of components you want
pca = PCA(n_components=num_components)
# Fit and transform the training data
X_train = pca.fit_transform(X_train_std)
# Transform the test data
X_test = pca.transform(X_test_std)
# Visualize the first two principal components
explained_var_ratio = pca.explained_variance_ratio_
# Scree plot
plt.figure(figsize=(10, 6))

```

```
plt.bar(range(1, num_components + 1), explained_var_ratio, alpha=0.7, align='center')
plt.title('Explained Variance Ratio for Each Principal Component')
plt.xlabel('Principal Component')
plt.ylabel('Explained Variance Ratio')
plt.show()
```

```
# Define the model
model = SGDClassifier(random_state=42)
```

```
# Define the hyperparameter grid
param_grid = {
    'alpha': [0.00001, 0.0001, 0.001, 0.01],
    'penalty': [None],
    'max_iter': [1000],
    'loss' : ['log_loss'],
    'learning_rate' : ['constant'],
    'eta0' : [0.0001, 0.001, 0.01, 0.1]
}
```

```
# Create GridSearchCV object with cross-validation
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)
```

```
# Get the best parameters and the corresponding model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_
```

```
all_models = grid_search.cv_results_['params']
```

```
# Use cross_val_score to get cross-validation scores for each model
cross_val_scores = []
for model_params in all_models:
    model = SGDClassifier(**model_params, random_state=42)
    scores = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy')
    cross_val_scores.append(scores)
```

```
plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)
plt.xticks(ticks=range(len(all_models)))
```

```
# Simulate partial_fit with batch-wise training
batch_size = X_train.shape[0]
num_samples = X_train.shape[0]
```

```

num_batches = int(np.ceil(num_samples / batch_size))
print("Best model : ", best_params)

# Training and validation losses
epochs = [60, 80]
for each in epochs:
    train_losses = []
    val_losses = []
    train_accuracies = []
    val_accuracies = []

    # best_sgd_model = SGDClassifier(**best_params)
    best_sgd_model = SGDClassifier(alpha=best_params['alpha'], eta0=best_params['eta0'],
                                    learning_rate=best_params['learning_rate'], loss=best_params['loss'],

max_iter=best_params['max_iter'], penalty=best_params['penalty'], random_state=42)

    # Learning rate schedule
    initial_learning_rate = 0.01
    learning_rate = initial_learning_rate

    # Early stopping parameters
    patience = 5
    best_val_loss = float('inf')
    no_improvement_count = 0

    for epoch in range(each):
        for batch_start in range(0, num_samples, batch_size):
            batch_end = min(batch_start + batch_size, num_samples)
            X_batch = X_train[batch_start:batch_end]
            y_batch = y_train[batch_start:batch_end]

            # Train the model on the batch
            best_sgd_model.partial_fit(X_batch, y_batch, classes=np.unique(y_train))
        # print(epoch)

    # Evaluate on training and validation sets
    train_predictions = best_sgd_model.predict(X_train)
    val_predictions = best_sgd_model.predict(X_test)
    train_probability = best_sgd_model.predict_proba(X_train)
    val_probability = best_sgd_model.predict_proba(X_test)
    # print(y_train)
    # print(train_predictions)
    train_loss = log_loss(y_train, train_probability)

```

```

val_loss = log_loss(y_test, val_probability)
train_losses.append(train_loss)
val_losses.append(val_loss)
train_accuracy = accuracy_score(y_train, train_predictions)
val_accuracy = accuracy_score(y_test, val_predictions)
train_accuracies.append(train_accuracy)
val_accuracies.append(val_accuracy)

## Learning rate schedule (decay every 10 epochs)
# if epoch % 10 == 0:
#     learning_rate = learning_rate * 0.9

# Plotting
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(range(1, len(train_losses) + 1), train_losses, label='Training Log Loss')
plt.plot(range(1, len(val_losses) + 1), val_losses, label='Validation Log Loss')
plt.title('Training and Validation Log Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Log Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(1, len(train_accuracies) + 1), train_accuracies, label='Training Accuracy')
plt.plot(range(1, len(val_accuracies) + 1), val_accuracies, label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.tight_layout()
plt.show()

# Evaluate the best model on the test set
y_pred = best_sgd_model.predict(X_test)
y_prob = best_sgd_model.decision_function(X_test)

# Testing accuracy
test_accuracy = accuracy_score(y_test, y_pred)
print("Testing Accuracy:", test_accuracy)

# Classification report
print("Classification Report:")

```

```

print(classification_report(y_test, y_pred))

# Confusion matrix
confusion_mat = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(confusion_mat)

```

3.3. SVM with Kernels

```

from tensorflow.keras.datasets import fashion_mnist
import matplotlib.pyplot as plt

# Load and preprocess the Fashion MNIST dataset
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()
X_train = X_train.reshape((60000, 28, 28)).astype('float32')
X_test = X_test.reshape((10000, 28, 28)).astype('float32')

class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(X_train[i], cmap=plt.cm.binary)
    plt.xlabel(class_names[y_train[i]])
plt.show()

import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
from sklearn.decomposition import PCA

# Load Fashion MNIST dataset
# fashion_mnist = datasets.fetch_openml('Fashion-MNIST', version=1, cache=True)
X_train = X_train.reshape(60000, 28*28)
X_test = X_test.reshape(10000, 28*28)

# Standardize the features

```

```

scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)

# Apply PCA for dimensionality reduction (optional)
# You can skip this step or adjust the number of components based on your needs
num_components = 200 # You can choose the number of components you want
pca = PCA(n_components=num_components)

# Fit and transform the training data
x_train_pca = pca.fit_transform(X_train_std)

# Transform the test data
x_test_pca = pca.transform(X_test_std)

# Visualize the first two principal components
explained_var_ratio = pca.explained_variance_ratio_

# Scree plot
plt.figure(figsize=(10, 6))
plt.bar(range(1, num_components + 1), explained_var_ratio, alpha=0.7, align='center')
plt.title('Explained Variance Ratio for Each Principal Component')
plt.xlabel('Principal Component')
plt.ylabel('Explained Variance Ratio')
plt.show()

# Initialize and train an SVM with RBF kernel
# svm_model = SVC(kernel='rbf', C=1.0, gamma='scale') # You can adjust hyperparameters like C and gamma
# svm_model.fit(X_train_pca, y_train)

# # Make predictions
# y_pred = svm_model.predict(X_test_pca)

# # Evaluate the model
# accuracy = accuracy_score(y_test, y_pred)
# print(f"Accuracy: {accuracy:.4f}")

# # Display additional metrics
# print("\nClassification Report:")
# print(classification_report(y_test, y_pred))

import matplotlib.pyplot as plt
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from sklearn.linear_model import SGDClassifier

```

```

from sklearn.kernel_approximation import RBFSampler
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score, classification_report, roc_curve, roc_auc_score,
hinge_loss
from sklearn.metrics import log_loss

import numpy as np

# Define the RBF Sampler
rbf_sampler = RBFSampler(gamma=0.01, random_state=42)

# Apply kernel approximation to the training and testing data
X_train_transformed = rbf_sampler.fit_transform(X_train)
X_test_transformed = rbf_sampler.transform(X_test)

# Define the model
model = SGDClassifier(random_state=42, )

# Define the hyperparameter grid
param_grid = {
    'alpha': [0.0001, 0.001],
    'penalty': ['l1', 'l2'],
    'max_iter': [1000],
    'loss' : ['hinge'],
    'learning_rate' : ['constant'],
    'eta0' : [0.0001, 0.001, 0.01]
}

# Create GridSearchCV object with cross-validation
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train_transformed, y_train)

# Get the best parameters and the corresponding model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split

all_models = grid_search.cv_results_['params']

# Use cross_val_score to get cross-validation scores for each model

```

```

cross_val_scores = []
for model_params in all_models:
    model = SGDClassifier(**model_params)
    scores = cross_val_score(model, X_train_transformed, y_train, cv=5, scoring='accuracy') # Fix the
variable name
    cross_val_scores.append(scores)

plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)
plt.xticks(ticks=range(len(all_models)), labels=['Model' + str(i+1) for i in range(len(all_models))],
rotation=45, ha='right')
plt.title('Cross-Validation Scores for Each Model')
plt.xlabel('Model Parameters')
plt.ylabel('Accuracy')
plt.show()

print('Cross validation scores : ', cross_val_scores)

# Simulate partial_fit with batch-wise training
batch_size = 32
num_samples = X_train_transformed.shape[0]
num_batches = int(np.ceil(num_samples / batch_size))

print("Best model : ", best_params)

# Training and validation losses
epochs = 60
train_losses = []
val_losses = []
train_accuracies = []
val_accuracies = []

for epoch in range(epochs):
    for batch_start in range(0, num_samples, batch_size):
        batch_end = min(batch_start + batch_size, num_samples)
        X_batch = X_train_transformed[batch_start:batch_end]
        y_batch = y_train[batch_start:batch_end]

        # Train the model on the batch
        best_model.partial_fit(X_batch, y_batch, classes=np.unique(y_train))

    # Evaluate on training and validation sets
    train_predictions = best_model.predict(X_train_transformed)
    val_predictions = best_model.predict(X_test_transformed)

```



```

train_loss = hinge_loss(y_train, best_model.decision_function(X_train_transformed))
val_loss = hinge_loss(y_test, best_model.decision_function(X_test_transformed))

train_losses.append(train_loss)
val_losses.append(val_loss)

train_accuracy = accuracy_score(y_train, train_predictions)
val_accuracy = accuracy_score(y_test, val_predictions)

train_accuracies.append(train_accuracy)
val_accuracies.append(val_accuracy)

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(range(1, epochs + 1), train_losses, label='Training Hinge Loss')
plt.plot(range(1, epochs + 1), val_losses, label='Validation Hinge Loss')
plt.title('Training and Validation Hinge Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Hinge Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(1, epochs + 1), train_accuracies, label='Training Accuracy')
plt.plot(range(1, epochs + 1), val_accuracies, label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.tight_layout()
plt.show()

# Evaluate the best model on the test set
y_pred = best_model.predict(X_test_transformed)
y_prob = best_model.decision_function(X_test_transformed)

# Testing accuracy
test_accuracy = accuracy_score(y_test, y_pred)
print("Testing Accuracy:", test_accuracy)

# Classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))

```

3.4. SVM without Kernels

```
from tensorflow.keras.datasets import fashion_mnist
import matplotlib.pyplot as plt

# Load and preprocess the Fashion MNIST dataset
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()
X_train = X_train.reshape((60000, 28, 28)).astype('float32')
X_test = X_test.reshape((10000, 28, 28)).astype('float32')

class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(X_train[i], cmap=plt.cm.binary)
    plt.xlabel(class_names[y_train[i]])
plt.show()

import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
from sklearn.decomposition import PCA

# Load Fashion MNIST dataset
# fashion_mnist = datasets.fetch_openml('Fashion-MNIST', version=1, cache=True)
X_train = X_train.reshape(60000, 28*28)
X_test = X_test.reshape(10000, 28*28)

# Standardize the features
scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)

# Apply PCA
pca = PCA(n_components=23) # You can adjust the explained variance threshold
X_train_pca = pca.fit_transform(sc_X_train)
X_test_pca = pca.transform(sc_X_test)
```

```

# Define the RBF Sampler
rbf_sampler = RBFSampler(gamma=0.001, random_state=42) # Initial gamma, you can change it later

# Define the model
model = SGDClassifier(random_state=42)

# Define the hyperparameter grid
param_grid = {
    'alpha': [0.0001, 0.01],
    'penalty': ['l1', 'l2'],
    'max_iter': [1000],
    'loss': ['hinge'],
    'learning_rate': ['constant'],
    'eta0': [0.0001, 0.001, 0.01]
}

X_train_transformed = X_train_pca
X_test_transformed = X_test_pca

# Create GridSearchCV object with cross-validation
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train_transformed, y_train)

# Get the best parameters and the corresponding model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split

all_models = grid_search.cv_results_['params']

# Use cross_val_score to get cross-validation scores for each model
cross_val_scores = []
for model_params in all_models:
    model = SGDClassifier(**model_params)
    scores = cross_val_score(model, X_train_transformed, y_train, cv=5, scoring='accuracy')
    cross_val_scores.append(scores)

plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)

```

```
plt.xticks(ticks=range(len(all_models)), labels=['Model' + str(i+1) for i in range(len(all_models))],
rotation=45, ha='right')
plt.title('Cross-Validation Scores for Each Model')
plt.xlabel('Model Parameters')
plt.ylabel('Accuracy')
plt.show()
```

```
# Simulate partial_fit with batch-wise training
batch_size = 32
num_samples = X_train_transformed.shape[0]
num_batches = int(np.ceil(num_samples / batch_size))
```

```
print("Best model : ", best_params)
```

```
# Training and validation losses
epochs = 60
train_losses = []
val_losses = []
train_accuracies = []
val_accuracies = []
```

```
for epoch in range(epochs):
    for batch_start in range(0, num_samples, batch_size):
        batch_end = min(batch_start + batch_size, num_samples)
        X_batch = X_train_transformed[batch_start:batch_end]
        y_batch = y_train[batch_start:batch_end]

        # Train the model on the batch
        best_model.partial_fit(X_batch, y_batch, classes=np.unique(y_train))
```

```
# Evaluate on training and validation sets
train_predictions = best_model.predict(X_train_transformed)
val_predictions = best_model.predict(X_test_transformed)
```

```
train_loss = hinge_loss(y_train, train_predictions)
val_loss = hinge_loss(y_test, val_predictions)
```

```
train_losses.append(train_loss)
val_losses.append(val_loss)
```

```
train_accuracy = accuracy_score(y_train, train_predictions)
val_accuracy = accuracy_score(y_test, val_predictions)
```

```
train_accuracies.append(train_accuracy)
```

```

val_accuracies.append(val_accuracy)

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(range(1, epochs + 1), train_losses, label='Training Hinge Loss')
plt.plot(range(1, epochs + 1), val_losses, label='Validation Hinge Loss')
plt.title('Training and Validation Hinge Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Hinge Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(1, epochs + 1), train_accuracies, label='Training Accuracy')
plt.plot(range(1, epochs + 1), val_accuracies, label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.tight_layout()
plt.show()

# Evaluate the best model on the test set
y_pred = best_model.predict(X_test_transformed)
y_prob = best_model.decision_function(X_test_transformed)

# Testing accuracy
test_accuracy = accuracy_score(y_test, y_pred)
print("Testing Accuracy:", test_accuracy)

# Classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))

# ROC curve and AUC score
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc_score = roc_auc_score(y_test, y_prob)

plt.figure(figsize=(8, 8))
plt.plot(fpr, tpr, label=f'AUC = {auc_score:.2f}')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.title('ROC Curve')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')

```

```
plt.legend()
plt.show()
```

3.5. PCA + k-NN

```
!pip install tensorflow
from sklearn.datasets import fetch_openml
from sklearn.preprocessing import StandardScaler
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import fashion_mnist
from sklearn.kernel_approximation import RBFSampler
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score, classification_report, roc_curve
from sklearn.metrics import roc_auc_score, log_loss, confusion_matrix
import seaborn as sns
import pandas as pd
from sklearn import datasets
from sklearn.decomposition import PCA
from ucimlrepo import fetch_ucirepo
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Load and preprocess the Fashion MNIST dataset
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()
X_train = X_train.reshape((60000, 28, 28)).astype('float32')
X_test = X_test.reshape((10000, 28, 28)).astype('float32')
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

# Load Fashion MNIST dataset
# fashion_mnist = datasets.fetch_openml('Fashion-MNIST', version=1, cache=True)
X_train = X_train.reshape(60000, 28*28)
X_test = X_test.reshape(10000, 28*28)
# Standardize the features
scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)

max_components = min(X_test.shape[0], X_test.shape[1])
print("Maximum number of components:", max_components)

num_components= [5, 15, 20]
```

```

for every in num_components:
    # Apply PCA for dimensionality reduction (optional)
    # You can skip this step or adjust the number of components based on your needs
    # num_components = 15 # You can choose the number of components you want
    pca = PCA(n_components=every)
    # Fit and transform the training data
    X_train_transformed = pca.fit_transform(X_train)
    # Transform the test data
    X_test_transformed = pca.transform(X_test)
    # Visualize the first two principal components
    explained_var_ratio = pca.explained_variance_ratio_
    # Scree plot
    plt.figure(figsize=(10, 6))
    plt.bar(range(1, every + 1), explained_var_ratio, alpha=0.7, align='center')
    plt.title('Explained Variance Ratio for Each Principal Component')
    plt.xlabel('Principal Component')
    plt.ylabel('Explained Variance Ratio')
    plt.show()

    # Define the model
    model = KNeighborsClassifier()

    # Define the hyperparameter grid
    param_grid = {
        'n_neighbors': [2,3,4,5,6,7,8],
        'weights': ['uniform','distance'],
        'p': [1,2]
    }

    # Create GridSearchCV object with cross-validation
    grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
    grid_search.fit(X_train_transformed, y_train)

    # Get the best parameters and the corresponding model
    best_params = grid_search.best_params_
    best_model = grid_search.best_estimator_

    all_models = grid_search.cv_results_['params']

    # Use cross_val_score to get cross-validation scores for each model
    cross_val_scores = []
    for model_params in all_models:
        model = KNeighborsClassifier(**model_params)
        scores = cross_val_score(model, X_train_transformed, y_train, cv=5, scoring='accuracy')

```

```

cross_val_scores.append(scores)

plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)
plt.xticks(ticks=range(len(all_models)))

# Simulate partial_fit with batch-wise training
batch_size = X_train.shape[0]
num_samples = X_train.shape[0]
num_batches = int(np.ceil(num_samples / batch_size))
print("Best model : ", best_params)

# best_sgd_model = SGDClassifier(**best_params)
best_knn_model = KNeighborsClassifier(n_neighbors=best_params['n_neighbors'])

# Train the model
best_knn_model.fit(X_train_transformed, y_train)

# Evaluate the best model on the test set
y_pred = best_knn_model.predict(X_test_transformed)
y_prob = best_knn_model.predict_proba(X_test_transformed)

# Testing accuracy
test_accuracy = accuracy_score(y_test, y_pred)
print("Testing Accuracy:", test_accuracy)

# Classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))

# Confusion matrix
confusion_mat = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(confusion_mat)

```

3.6. FNN

```

from tensorflow.keras.datasets import fashion_mnist
import matplotlib.pyplot as plt

# Load and preprocess the Fashion MNIST dataset
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

```



```
X_train = X_train.reshape((60000, 28, 28)).astype('float32')
X_test = X_test.reshape((10000, 28, 28)).astype('float32')
```

```
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```

```
plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(X_train[i], cmap=plt.cm.binary)
    plt.xlabel(class_names[y_train[i]])
plt.show()
```

```
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
from sklearn.decomposition import PCA
```

```
# Load Fashion MNIST dataset
# fashion_mnist = datasets.fetch_openml('Fashion-MNIST', version=1, cache=True)
X_train = X_train.reshape(60000, 28*28)
X_test = X_test.reshape(10000, 28*28)
```

```
# Standardize the features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
import torch.nn as nn
import torch
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score
```

```
input_size = 784
output_size = 10
```

```
num_epochs = 50
batch_size = 32
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
# Assuming X_train, y_train, X_test, and y_test are NumPy arrays
X_train_tensor = torch.from_numpy(X_train).to(device).float()
y_train_tensor = torch.from_numpy(y_train).to(device).long()
X_test_tensor = torch.from_numpy(X_test).to(device).float()
y_test_tensor = torch.from_numpy(y_test).to(device).long()
```

```
class FNN(nn.Module):
    def __init__(self, input_size, hidden_sizes, output_size, dropout_prob=0.4):
        super(FNN, self).__init__()
        layer_sizes = [input_size] + hidden_sizes + [output_size]
        layers = []
        for i in range(len(layer_sizes)-1):
            layers.append(nn.Linear(layer_sizes[i], layer_sizes[i + 1]))
            if i < len(layer_sizes) - 2:
                layers.append(nn.ReLU()) # Adding ReLU activation between hidden layers
        self.model = nn.Sequential(*layers)

    def forward(self, x):
        return self.model(x)
```

```
from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from tqdm import tqdm
import torch.nn.functional as F
```

```
param_grid = {
    'lr': [0.001, 0.01, 0.1],
    'hidden_sizes': [[128, 64, 8], [256, 16, 8], [128, 16], [256, 32]],
    'input_size': [X_train.shape[1]],
    'output_size': [10],
    'criterion': [nn.CrossEntropyLoss()],
    'optimizer_class': [optim.Adam],
    'wd': [0.001, 0.01],
    'num_epochs': [50, 80],
    'batch_size': [X_train.shape[0]],
```

```
'model_class' : [FNN]
}
```

```
# Custom PyTorch estimator for GridSearchCV
```

```
class PyTorchEstimator(BaseEstimator, ClassifierMixin):
```

```
    def __init__(self, model_class, criterion, optimizer_class, input_size=30, hidden_sizes = [32,16],
output_size = 10, lr=0.001, wd=0.0, num_epochs=20, batch_size=32, model = None):
```

```
        self.model_class = model_class
```

```
        self.criterion = criterion
```

```
        self.optimizer_class = optimizer_class
```

```
        self.input_size = input_size
```

```
        self.hidden_sizes = hidden_sizes
```

```
        self.output_size = output_size
```

```
        self.lr = lr
```

```
        self.wd = wd
```

```
        self.num_epochs = num_epochs
```

```
        self.batch_size = batch_size
```

```
        self.model = None
```

```
def fit(self, X, y):
```

```
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
    X_tensor = torch.from_numpy(X).to(device).float()
```

```
    y_tensor = torch.from_numpy(y).to(device).long()
```

```
    num_samples = X_tensor.shape[0]
```

```
    self.model = self.model_class(self.input_size, self.hidden_sizes, self.output_size).to(device)
```

```
    #print(self.model)
```

```
    optimizer = self.optimizer_class(self.model.parameters(), lr=self.lr, weight_decay=self.wd)
```

```
    for epoch in range(self.num_epochs):
```

```
        self.model.train()
```

```
        #print(epoch)
```

```
        for batch_start in range(0, num_samples, self.batch_size):
```

```
            batch_end = min(batch_start + self.batch_size, num_samples)
```

```
            X_batch = X_tensor[batch_start:batch_end]
```

```
            y_batch = y_tensor[batch_start:batch_end]
```

```
            optimizer.zero_grad()
```

```
            y_pred = self.model(X_batch)
```

```
            loss = nn.CrossEntropyLoss()(y_pred.cpu(), y_batch.cpu())
```

```
            loss.backward()
```

```

        optimizer.step()

    if epoch % 10 == 0:
        print(f'Epoch {epoch + 1} : Parameters - Batch size: {self.batch_size}, lr: {self.lr}, wd: {self.wd}')

    return self

def predict(self, X):
    self.model.eval()
    with torch.no_grad():
        X_tensor = torch.from_numpy(X).to(device).float()
        output = self.model(X_tensor).cpu()
        #print(torch.argmax(output, 1).numpy())
    return torch.argmax(output, 1).numpy()

def get_params(self, deep=True):
    return {
        'model_class': self.model_class,
        'criterion': self.criterion,
        'optimizer_class': self.optimizer_class,
        'input_size': self.input_size,
        'hidden_sizes': self.hidden_sizes,
        'output_size': self.output_size,
        'lr': self.lr,
        'num_epochs': self.num_epochs,
        'batch_size': self.batch_size,
        'model': self.model,
        'wd': self.wd
    }

pytorch_estimator = PyTorchEstimator(FNN, nn.CrossEntropyLoss(), optim.Adam)
grid_search = GridSearchCV(pytorch_estimator, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)

best_params = grid_search.best_params_

import seaborn as sns
import matplotlib.pyplot as plt

all_models = grid_search.cv_results_['params']

```

```

# Use cross_val_score to get cross-validation scores for each model
cross_val_scores = []
for model_params in all_models:
    model = PyTorchEstimator(**model_params)
    scores = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy')
    cross_val_scores.append(scores)

plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)
plt.xticks(ticks=range(len(all_models)), labels=['Model' + str(i+1) for i in range(len(all_models))],
rotation=45, ha='right')
plt.title('Cross-Validation Scores for Each Model')
plt.xlabel('Model Parameters')
plt.ylabel('Accuracy')
plt.show()

X_train = torch.from_numpy(X_train).to(device).float()
y_train = torch.from_numpy(y_train).to(device).long()
X_test = torch.from_numpy(X_test).to(device).float()
y_test = torch.from_numpy(y_test).to(device).long()

import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, classification_report, roc_curve, roc_auc_score,
confusion_matrix
import matplotlib.pyplot as plt

num_samples = X_train.shape[0]

model = best_params['model_class'](best_params['input_size'], best_params['hidden_sizes'],
best_params['output_size']).to(device)
optimizer = best_params['optimizer_class'](model.parameters(), lr=best_params['lr'], weight_decay =
best_params['wd'])

# Training and validation losses
epochs = 50
train_losses = []
val_losses = []
train_accuracies = []
val_accuracies = []

for epoch in range(best_params['num_epochs']):
    model.train()
    for batch_start in range(0, num_samples, num_samples):

```

```
batch_end = min(batch_start + num_samples, num_samples)
X_batch = X_train[batch_start:batch_end]
y_batch = y_train[batch_start:batch_end]
```

```
optimizer.zero_grad()
y_pred = model(X_train)
#print(y_pred)
#print(y_pred,y_batch.view(-1, 1))
```

```
loss = best_params['criterion'](y_pred.to(device), y_train)
loss.backward()
```

```
optimizer.step()
```

```
# Evaluate on training and validation sets
```

```
model.eval()
with torch.no_grad():
    train_predictions = torch.argmax(model(X_train).cpu(), 1)
    val_predictions = torch.argmax(model(X_test).cpu(), 1)
```

```
# Compute loss and accuracy
```

```
train_loss = best_params['criterion'](model(X_train).cpu(),y_train.cpu())
val_loss = best_params['criterion'](model(X_test).cpu(), y_test.cpu())
```

```
train_losses.append(train_loss.item())
val_losses.append(val_loss.item())
```

```
train_accuracy = accuracy_score(y_train.cpu().numpy(), train_predictions.cpu().numpy())
val_accuracy = accuracy_score(y_test.cpu().numpy(), val_predictions.cpu().numpy())
```

```
train_accuracies.append(train_accuracy)
val_accuracies.append(val_accuracy)
```

```
#print(val_accuracy)
```

```
print(f'Epoch : {epoch+1}, Training Loss : {train_loss}, Validation Loss : {val_loss}')
```

```
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(range(1, best_params['num_epochs'] + 1), train_losses, label='Training Hinge Loss')
plt.plot(range(1, best_params['num_epochs'] + 1), val_losses, label='Validation Hinge Loss')
plt.title('Training and Validation Hinge Loss Over Epochs')
plt.xlabel('Epoch')
```

```

plt.ylabel('Hinge Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(1, best_params['num_epochs'] + 1), train_accuracies, label='Training Accuracy')
plt.plot(range(1, best_params['num_epochs'] + 1), val_accuracies, label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.tight_layout()
plt.show()

# Evaluate the best model on the test set
model.eval()

# Predictions and decision function (sigmoid output)
with torch.no_grad():
    #print(model(X_test))
    y_prob_tensor = model(X_test)
    y_pred_tensor = torch.argmax(y_prob_tensor.cpu(), 1).numpy()

# Convert PyTorch tensors to NumPy arrays
y_pred = y_pred_tensor
y_prob = y_prob_tensor

# Testing accuracy
test_accuracy = accuracy_score(y_test.cpu().detach().numpy(), y_pred)
print("Testing Accuracy:", test_accuracy)

# Classification report
print("Classification Report:")
print(classification_report(y_test.cpu().detach().numpy(), y_pred))

```

3.7. CNN

```

import numpy as np
from tensorflow.keras.datasets import fashion_mnist

(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
x_train = x_train.reshape((60000, 28, 28)).astype('float32')
x_test = x_test.reshape((10000, 28, 28)).astype('float32')

```

```

num_images = x_train.shape[0]
num_pixels = x_train.shape[1] * x_train.shape[2]
X_train_flattened = x_train.reshape(num_images, num_pixels)

mean = np.mean(X_train_flattened, axis=0)
std = np.std(X_train_flattened, axis=0)

epsilon = 1e-10
X_train_normalized = (X_train_flattened - mean) / (std + epsilon)

X_train_normalized = X_train_normalized.reshape(num_images, x_train.shape[1], x_train.shape[2])

normalized_mean = np.mean(X_train_normalized)
normalized_std = np.std(X_train_normalized)
print("Mean of normalized data:", normalized_mean)
print("Standard deviation of normalized data:", normalized_std)

num_images_test = x_test.shape[0]
X_test_flattened = x_test.reshape(num_images_test, -1) # Flattening keeping the number of pixels per
image

mean_test = np.mean(X_test_flattened, axis=0)
std_test = np.std(X_test_flattened, axis=0)

X_test_normalized = (X_test_flattened - mean) / (std + epsilon)

X_test_normalized = X_test_normalized.reshape(num_images_test, x_test.shape[1], x_test.shape[2])

normalized_mean_test = np.mean(X_test_normalized)
normalized_std_test = np.std(X_test_normalized)
print("Mean of normalized test data:", normalized_mean_test)
print("Standard deviation of normalized test data:", normalized_std_test)

x_train = X_train_normalized
x_test = X_test_normalized

x_train = x_train.reshape(x_train.shape[0], 1, 28, 28)
x_test = x_test.reshape(x_test.shape[0], 1, 28, 28)

```



```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
import numpy as np
```

```
class_masks = [y_train == class_label for class_label in range(10)]
```

```
selected_indices = []
```

```
for class_mask in class_masks:
```

```
    class_indices = np.where(class_mask)[0]
```

```
    selected_indices.extend(np.random.choice(class_indices, size=1000, replace=False))
```

```
x_train = x_train[selected_indices]
```

```
y_train = y_train[selected_indices]
```

```
x_train[0]
```

```
plt.imshow(x_train[9].squeeze(), cmap="gray")
```

```
x_train[0]
```

```
plt.imshow(x_train[67].squeeze(), cmap="gray")
```

```
print(x_train[67])
```

```
# import torch.nn.functional as F
```

```
import torch.nn as nn
```

```
class CNN(nn.Module):
```

```
    def __init__(self, input_size, hidden_sizes, output_size, kernel_size_conv, kernel_size_pool,  
padding, stride_conv, stride_pool, dropout_prob=0.4):
```

```
        super(CNN, self).__init__()
```

```
        layers = []
```

```
        layers.append(nn.Conv2d(1, hidden_sizes[0], kernel_size=kernel_size_conv, padding=padding,  
stride=stride_conv))
```

```
        layers.append(nn.BatchNorm2d(hidden_sizes[0]))
```

```
        layers.append(nn.ReLU())
```

```
        layers.append(nn.MaxPool2d(kernel_size=kernel_size_pool, stride=stride_pool))
```

```
        for i in range(len(hidden_sizes) - 1):
```

```

        layers.append(nn.Conv2d(hidden_sizes[i], hidden_sizes[i+1], kernel_size=kernel_size_conv,
padding=0, stride=stride_conv))
        layers.append(nn.BatchNorm2d(hidden_sizes[i+1]))
        layers.append(nn.ReLU())
        layers.append(nn.MaxPool2d(kernel_size=kernel_size_pool, stride=stride_pool))

```

```

self.model = nn.Sequential(*layers)
self.flatten = nn.Flatten()
if hidden_sizes[-1] == 48:
    val = 6
else:
    val = 6
self.layer1 = nn.Linear(hidden_sizes[-1] * val * val, 600)
#self.layer2 = nn.Linear(600, 120)
self.layer3 = nn.Linear(600, output_size)
self.drop = nn.Dropout(dropout_prob)
self.softmax = nn.Softmax(dim=1)

```

```

def forward(self, x):
    output = self.model(x)
    #print(output.shape)
    output = self.flatten(output)
    #print(output.shape)
    output = F.relu(self.layer1(output))
    output = self.drop(output)
    # output = F.relu(self.layer2(output))
    # output = self.drop(output)
    output = self.layer3(output)

    #output = self.softmax(output)
    #print(output)

    return output

```

```

from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.model_selection import GridSearchCV
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from tqdm import tqdm

```

```

param_grid = {

```

```

'lr': [0.001],
'hidden_sizes': [[32, 48],[32, 64]],
'input_size': [28],
'output_size': [10],
'criterion': [nn.CrossEntropyLoss()],
'optimizer_class': [optim.Adam],
'wd': [0.001, 0.01],
'num_epochs': [5],
'batch_size': [x_train.shape[0]//2],
'model_class': [CNN],
'kernel_size_conv' : [3],
'kernel_size_pool' : [2],
'padding' : [1],
'stride_conv' : [1],
'stride_pool' : [2]

}

```

```

class PyTorchEstimator(BaseEstimator, ClassifierMixin):

```

```

    def __init__(self, model_class, criterion, optimizer_class, input_size=28, hidden_sizes=[32, 16],
output_size=10,

```

```

        lr=0.001, wd=0.0, num_epochs=20, batch_size=32, kernel_size_conv = 5, kernel_size_pool
= 2, padding = 1, stride_conv = 1, stride_pool = 1,model=None):

```

```

        self.model_class = model_class
        self.criterion = criterion
        self.optimizer_class = optimizer_class
        self.input_size = input_size
        self.hidden_sizes = hidden_sizes
        self.output_size = output_size
        self.lr = lr
        self.wd = wd
        self.num_epochs = num_epochs
        self.batch_size = batch_size
        self.model = None
        self.kernel_size_conv = kernel_size_conv
        self.kernel_size_pool = kernel_size_pool
        self.padding = padding
        self.stride_conv = stride_conv
        self.stride_pool = stride_pool

```

```

    def fit(self, X, y):

```

```

        train_dataset = TensorDataset(torch.from_numpy(X).float(), torch.from_numpy(y))
        train_loader = DataLoader(train_dataset, batch_size=self.batch_size, shuffle=True)

```

```

num_samples = len(train_loader.dataset)

self.model = self.model_class(self.input_size, self.hidden_sizes, self.output_size,
self.kernel_size_conv, self.kernel_size_pool, self.padding, self.stride_conv, self.stride_pool).to(device)
optimizer = self.optimizer_class(self.model.parameters(), lr=self.lr)

for epoch in range(self.num_epochs):
    self.model.train()
    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)

        optimizer.zero_grad()
        y_pred = self.model(X_batch)
        #print(y_pred)
        loss = self.criterion(y_pred, y_batch)
        loss.backward()
        optimizer.step()
        print(loss.item())

    if epoch % 10 == 0:
        print(f'Epoch {epoch + 1} : Parameters - Batch size: {self.batch_size}, lr: {self.lr}, wd:
{self.wd}')

    # Free up GPU memory
    torch.cuda.empty_cache()

return self

def predict(self, X):
    self.model.eval()
    with torch.no_grad():
        X_tensor = torch.from_numpy(X).to(device).float()
        output = self.model(X_tensor).cpu()
        #print(torch.argmax(output, 1).numpy(), torch.argmax(output, 1).numpy().shape)

    return torch.max(output.data, 1)[1]

def get_params(self, deep=True):
    return {
        'model_class': self.model_class,
        'criterion': self.criterion,
        'optimizer_class': self.optimizer_class,
        'input_size': self.input_size,
        'hidden_sizes': self.hidden_sizes,

```

```

        'output_size': self.output_size,
        'lr': self.lr,
        'num_epochs': self.num_epochs,
        'batch_size': self.batch_size,
        'model': self.model,
        'wd': self.wd,
        'kernel_size_conv' : self.kernel_size_conv,
        'kernel_size_pool' : self.kernel_size_pool,
        'padding' : self.padding,
        'stride_conv' : self.stride_conv,
        'stride_pool' : self.stride_pool
    }

```

```

# print(x_train[0], x_train.shape)
# Instantiate the PyTorchEstimator with CNN model
pytorch_estimator = PyTorchEstimator(CNN, nn.CrossEntropyLoss(), optim.Adam)
grid_search = GridSearchCV(pytorch_estimator, param_grid, cv=5, scoring='accuracy')
grid_search.fit(x_train, y_train)

```

```

best_params = grid_search.best_params_

```

```

# Free up GPU memory after the grid search
torch.cuda.empty_cache()

```

```

import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split

```

```

all_models = grid_search.cv_results_['params']

```

```

# Use cross_val_score to get cross-validation scores for each model
cross_val_scores = []
for model_params in all_models:
    model = PyTorchEstimator(**model_params)
    scores = cross_val_score(model, x_train, y_train, cv=5, scoring='accuracy')
    print(scores)
    cross_val_scores.append(scores)

```

```

plt.figure(figsize=(12, 8))
sns.boxplot(data=cross_val_scores)
plt.xticks(ticks=range(len(all_models)), labels=['Model' + str(i+1) for i in range(len(all_models))],
rotation=45, ha='right')
plt.title('Cross-Validation Scores for Each Model')

```

```
plt.xlabel('Model Parameters')
plt.ylabel('Accuracy')
plt.show()
```

```
torch.save(best_params, 'saved_model.pth')
```

```
x_train = (x_train).to(device).float()
y_train = (y_train).to(device).long()
```

```
x_train = torch.tensor(x_train)
y_train = torch.tensor(y_train)
```

```
x_test = torch.tensor(x_test)
y_test = torch.tensor(y_test)
```

```
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, classification_report, roc_curve, roc_auc_score,
confusion_matrix
import matplotlib.pyplot as plt
```

```
model = best_params['model_class'](best_params['input_size'], best_params['hidden_sizes'],
best_params['output_size'], best_params['kernel_size_conv'], best_params['kernel_size_pool'],
best_params['padding'], best_params['stride_conv'], best_params['stride_pool']).to(device)
optimizer = best_params['optimizer_class'](model.parameters(), lr=best_params['lr'], weight_decay =
best_params['wd'])
```

```
num_samples = x_train.shape[0]
```

```
# Training and validation losses
epochs = 20
train_losses = []
val_losses = []
train_accuracies = []
val_accuracies = []
```

```
for epoch in range(best_params['num_epochs']):
    model.train()
    for batch_start in range(0, num_samples, 32):
        batch_end = min(batch_start + 32, num_samples)
        X_batch = x_train[batch_start:batch_end]
        y_batch = y_train[batch_start:batch_end]

        optimizer.zero_grad()
```

```

y_pred = model(x_train.to(device))
#print(y_pred)
#print(y_pred,y_batch.view(-1, 1))

loss = best_params['criterion'](y_pred.to(device), y_train.to(device))
loss.backward()

optimizer.step()

# Evaluate on training and validation sets
model.eval()
with torch.no_grad():
    train_predictions = torch.argmax(model(x_train.to(device)).cpu(), 1)
    val_predictions = torch.argmax(model(x_test.to(device)).cpu(), 1)

# Compute loss and accuracy
train_loss = best_params['criterion'](model(x_train.to(device)).cpu(),y_train.cpu())
val_loss = best_params['criterion'](model(x_test.to(device)).cpu(), y_test.cpu())

train_losses.append(train_loss.item())
val_losses.append(val_loss.item())

train_accuracy = accuracy_score(y_train.cpu().numpy(), train_predictions.cpu().numpy())
val_accuracy = accuracy_score(y_test.cpu().numpy(), val_predictions.cpu().numpy())

train_accuracies.append(train_accuracy)
val_accuracies.append(val_accuracy)

#print(val_accuracy)

print(f'Epoch : {epoch+1}, Training Loss : {train_loss}, Validation Loss : {val_loss}')

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(range(1, best_params['num_epochs'] + 1), train_losses, label='Training Hinge Loss')
plt.plot(range(1, best_params['num_epochs'] + 1), val_losses, label='Validation Hinge Loss')
plt.title('Training and Validation Hinge Loss Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Hinge Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(1, best_params['num_epochs'] + 1), train_accuracies, label='Training Accuracy')

```

```
plt.plot(range(1, best_params['num_epochs'] + 1), val_accuracies, label='Validation Accuracy')
plt.title('Training and Validation Accuracy Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
```

```
plt.tight_layout()
plt.show()
```

```
# Evaluate the best model on the test set
model.eval()
```

```
# Predictions and decision function (sigmoid output)
with torch.no_grad():
    #print(model(X_test))
    y_prob_tensor = model(x_test.to(device))
    y_pred_tensor = torch.argmax(y_prob_tensor.cpu(), 1).numpy()
```

```
# Convert PyTorch tensors to NumPy arrays
y_pred = y_pred_tensor
y_prob = y_prob_tensor
```

```
# Testing accuracy
test_accuracy = accuracy_score(y_test.cpu().detach().numpy(), y_pred)
print("Testing Accuracy:", test_accuracy)
```

```
# Classification report
print("Classification Report:")
print(classification_report(y_test.cpu().detach().numpy(), y_pred))
```